FATGRAM

Integrantes

Pablo Sainz de la Maza Rodríguez

Gabriel Calvo Aja

Yeray Gutiérrez Mullor

Índice

1. Introducción 6
2. Objetivos 6
3. Análisis del contexto y estado del arte 7
1. Oportunidades de negocio y necesidades detectadas 7
2. Obligaciones legales 8
3. Obligaciones fiscales 8
4. Obligaciones laborales 8
5. Prevención de Riesgos Laborales 8
6. Ayudas y subvenciones 8
4. Requisitos 9
1. Requisitos Principales 9
1. Identificación de Necesidades del Cliente 9
2. Propuesta 10
3. Innovación y viabilidad técnica 11
3.1 Innovación 11
3.2 Viabilidad técnica 11
4.1 Requisitos técnicos 11
5. Diseño 12
a) Una Página Web, desarrollada con WordPress. 12
Desarrollo del Proceso de creación 12
b) Una infraestructura desarrollada en AWS segura, escalable y altamente disponible. 14
c) Una Base de datos de MariaDB en Amazon RDS. 16
d) Implementación de Ansible y Terraform. 16
e) Script de Monitorización de servicios automatizado con Cron. 17
f) Configuración e Implantación de un Proxy Inverso. 19
g) Diagrama de Infraestructura AWS 20
6. Planificación 21
Infraestructura de AWS 21
VPC 21
Subredes 21

Tablas de rutas 21
Internet Gateway y NAT 22
Grupos de seguridad 22
Instancias EC2 22
Base de datos en Amazon RDS 23
Proxy inverso y balanceo de carga 23
Servidores web y WordPress 23
Configuración de la base de datos en WordPress 24
PHPMyAdmin 24
Automatización con Terraform y Ansible 24
Terraform 24
Ansible 24
Monitorización 25
Migración 25
Pruebas finales 25
7. Implementación 26
Instancia NAT: 26
Grupos de Seguridad: 26
Grupo de Seguridad de la Instancia NAT: 27
Comandos en la Instancia NAT: 29
Infraestructura de AWS 30
Configuración de la VPC 30
Configuración de las subredes 31
Configuración de las tablas de enrutamiento 31
Subred pública 31
Subred privada 31
Configuración de las instancias 32
Descripción 32
Grupos de seguridad 32
Descripción 33
Configuración de la base de datos RDS 34
Proxy en Instancia NAT de AWS: 35

Configuración en Nginx 35
Resultado: 36
WordPress: 38
Instalar nginx: 38
Instalar php y mysql: 38
Crear archivo de configuración en Nginx: 38
Modificar el archivo de configuración: 38
1. Escucha de tráfico (Puerto 80) 40
2. Directorio raíz y archivos índice 40
3. Manejo de URLs (Enlaces permanentes) 40
4. Procesamiento de PHP (El motor) 40
Instalar WordPress: 41
Resultado: 42
PHPMyAdmin: 44
Descargar él .zip de PHPMyAdmin: 44
Archivo de configuración de PHPMyAdmin: 44
Crear usuario para PHPMyAdmin: 45
Interfaz Web: 46
Migración: 47
Laboratorios: 47
1. Crear AMIs: 47
2. Compartir Instancias: 48
3. Lanzar las Instancias desde el nuevo laboratorio: 48
Resultado: 49
Crear una instantánea del Amazon RDS: 49
Cambiar KMS: 50
Realizar copia de la instantánea: 53
Compartir la nueva instantánea: 54
Copiar la instantánea en el nuevo servidor: 54
Restaurar el RDS: 55
RDS migrado: 56
Monitorización 57

Creación de Usuario Remoto 57
Fichero de Configuración 58
Pruebas de Conexiones SSH 59
Creación y configuración del Script 61
Configuración de CRON 70
8. Seguimiento 71
9. Conclusiones y Valoración Personal 73
Valoración del trabajo en grupo 73
Reflexión final del proyecto 74
Dificultades encontradas 74
Calidad del trabajo realizado y aprendizajes 74
Qué mejoraríamos en futuras ocasiones 74
10. Bibliografía y Anexos 75

1. Introducción

Nuestro proyecto se basará en una red social creada para compartir platos de comida, funcionará como Instagram. Estará dirigido especialmente a especialistas en la cocina, restaurantes que estén comenzando y amantes de la gastronomía

Dentro de esta aplicación, se usará para publicar fotos sobre comida y dar la opción de incluir la receta de dicha comida en la que se incluirán los valores nutritivos de los alimentos para demostrar si es bueno o malo para la salud. También, creemos que debería tener una tienda que permitirá comprar alimentos o pedir un plato a domicilio. Tendrá un espacio para mostrar restaurantes y ver sus menús y opinar sobre ellos.

Creemos que es importante porque con esta aplicación se puede dar voz a otros restaurantes que están empezando e impulsar su negocio o simplemente mostrar los platos que hacen otras personas y permitir que otros puedan replicarlos. Esta aplicación mostraría la receta y el valor nutritivo de cada plato para que se muestre las calorías de cada plato y se pueda comprobar si es mejor o peor para la salud.

2. Objetivos

- Objetivo general

Crear una red social para compartir platos de comida, parecida a Instagram, donde cocineros, restaurantes nuevos y amantes de la gastronomía puedan mostrar sus recetas, conocer el valor nutritivo de los platos y descubrir nuevos lugares para comer.

- Objetivos específicos
    - Permitir que los usuarios publiquen fotos de comida y añadan la receta de cada plato manteniendo la privacidad de sus datos teniendo una base de datos con la que almacenar sus datos como el correo electrónico asociado a la cuenta, número de teléfono, nombre y apellidos, nombre de usuario etc.
    - Mostrar la información nutricional y las calorías de los alimentos para que cualquiera pueda saber si un plato es más o menos saludable.
    - Incluir una tienda dentro de la app para comprar ingredientes o pedir comida a domicilio a través de compras online.
    - Ofrecer un espacio donde los restaurantes puedan mostrar sus menús y recibir opiniones de los usuarios teniendo cada restaurante su propio blog utilizando jekyll.

- Dar visibilidad a restaurantes que están empezando y ayudar a impulsar sus negocios mostrando la ubicación en Google Maps integrado en la página.
- Facilitar que las personas descubran nuevas recetas y puedan replicar los platos que ven en la app mediante recetas o videos para que la gente pueda enseñar a hacer sus platos.

3. Análisis del contexto y estado del arte

Para la creación de nuestra empresa, necesitamos un sitio web, bases de datos, una red en la nube, asegurar los datos de la empresa y que el sitio esté 24/7 activo.

En cuanto a nuestro sitio web de la aplicación, usaremos Github Pages para subir la página web, aunque se podría usar Cloudflare o Netlify y para el sitio web de la empresa, vamos a usar nginx dentro de AWS para hacer el sitio de la empresa

Para las bases de datos, pensamos usar MySQL, pero también tenemos pensado usar MongoDB o MariaDB

Para la infraestructura en la nube, usaremos AWS y cabe la posibilidad de cambiar a Microsoft Azure.

Como hemos mencionado previamente en la introducción, nuestra aplicación tiene parte de la funcionalidad de Instagram en el apartado de fotografías. En cuanto a temática, tendría una similitud a JustEat o Glovo para el servicio a domicilio.

A diferencia de otros, nuestro proyecto aborda el anuncio de restaurantes y su valoración, junto a esto permitirá el servicio a domicilio de platos estos restaurantes. Por otra parte, el cliente podrá crear los platos gracias a la publicación de las recetas, así como modificarla a gusto del usuario para cambiar algún ingrediente y que sea más o menos saludable a cambio de sabor.

1. Oportunidades de negocio y necesidades detectadas

Fatgram surge como una red social especializada en gastronomía, combinando publicación de platos, recetas, análisis nutricional y visibilidad para restaurantes emergentes. La propuesta responde a varias necesidades:

- Visibilidad para restaurantes pequeños.

- Acceso a recetas e información nutricional para usuarios.
- Marketplace integrado para pedidos y compra de ingredientes.
- Tendencia creciente hacia alimentación saludable y contenido visual interactivo.

2. Obligaciones legales

- Protección de datos: consentimiento explícito, políticas de privacidad, seguridad de la información.
- Propiedad intelectual: los usuarios conservan derechos sobre sus fotos, pero se ceden a Fatgram para publicación.
- Condiciones de uso y comercio electrónico (LSSI): normas de publicación, compras, devoluciones y pagos.
- Prevención de fraude en pagos electrónicos (PSD2).

3. Obligaciones fiscales

- Impuesto de Sociedades (25%) si se constituye SL.
- IVA (21%) sobre servicios de publicidad y venta de productos.
- Declaraciones trimestrales y anuales, retenciones según convenios y modelo 347 si hay marketplace.

4. Obligaciones laborales

- Contratos conforme al Estatuto de los Trabajadores.
- Seguridad Social, nóminas, control horario y prevención de riesgos laborales.
- Si hay subcontrataciones, contratos mercantiles y cumplimiento RGPD.

5. Prevención de Riesgos Laborales

- Ergonomía de puestos de trabajo.
- Equipos seguros y formación básica en PRL.
- Plan de emergencia y vigilancia de la salud.

6. Ayudas y subvenciones

- Kit Digital y programas NEOTEC / ENISA para startups tecnológicas.

- Subvenciones autonómicas para proyectos TIC.
- Programas europeos (Horizon Europe) si hay innovación en nutrición o IA.
- Bonificaciones y ayudas para jóvenes emprendedores.

4. Requisitos

1. Requisitos Principales

- Una Página Web, desarrollada con WordPress.

- Manejo y Control de AWS.

- Una infraestructura segura, escalable y altamente segura y disponible.

- Una Base de datos en MariaDB.

- Implementación de Ansible y Terraform.

- Script de Monitorización de servicios automatizado con Cron.

- Configuración e Implantación de un Proxy Inverso.

1. Identificación de Necesidades del Cliente

El "cliente" en este caso son dos perfiles: el **usuario final** (usuario/cocinero amateur) y el chef del restaurante.

a. Necesidades de los Usuarios

- **Centralización:** Evitar saltar de una app para buscar recetas a otra para reservar mesa.
- **Filtros Inteligentes:** Necesidad de buscar por ingredientes disponibles, restricciones dietéticas (celiacos o diabéticos) o presupuesto.
- **Interacción Real:** Más allá del "like", el usuario busca feedback sobre si una receta realmente sale bien o si el servicio de un restaurante es constante.

b. Necesidades de los Restaurantes

- **Visibilidad Segmentada:** Llegar a personas que realmente buscan el tipo de cocina del restaurante
- **Conversión Directa:** Que el descubrimiento de un plato se convierta en una reserva o un pedido a domicilio de forma inmediata.

2. Propuesta

El "Hub" Gastronómico Integral

La propuesta no es solo "compartir", sino **"gestionar la experiencia alimentaria"**. Para ello, Fatgram se divide en tres pilares operativos:

A. El Motor de Recomendación Hiper-Personalizado

A diferencia de otras redes, Fatgram utiliza un algoritmo de **Grafos de Sabor**. Si un usuario suele dar "like" a recetas sin gluten y busca restaurantes de sushi, la app no solo le muestra más sushi, sino que identifica patrones de ingredientes (ej. soja, arroz, pescado crudo) para sugerirle recetas de cocina peruana (Nikkei) que cumplan con sus restricciones.

B. El Módulo de "Cocina Activa"

Las recetas no son estáticas. La solución incluye:

- **Modo Cocina Guiada:** Temporizadores integrados en los pasos de la receta y control por voz para no tocar el móvil con las manos sucias.
- **Conversor de Porciones Automático:** Ajuste instantáneo de las cantidades de la receta según el número de comensales.

C. Integración de Reserva y Pedido "In-App"

Fatgram actúa como un agregador. Si un usuario ve un video (Reel) de un *entrecot* flambeado en un restaurante local, la solución permite:

1. **Reservar mesa** en ese instante.
2. **Pre-pedir el plato** para que esté listo al llegar (optimizando tiempos del restaurante).

La solución a estas necesidades es sobretodo por el hecho de que las personas invertimos mucho más tiempo eligiendo qué vamos a comer que cenando. Por eso, buscamos que si el usuario encuentra una receta que tiene muchas ganas de probar, la aplicación pueda mostrarle restaurantes que sirvan este plato cerca del usuario. De esta forma, el usuario y el restaurante se benefician entre sí, en la que el usuario se beneficia del producto del restaurante, y el restaurante obtiene la visita y la economía del usuario y su posible recomendación

3. Innovación y viabilidad técnica

3.1 Innovación

Para diferenciar a Fatgram de la competencia, propongo los siguientes aspectos innovadores:

- **Escaneado de Recetas por IA:** Una función donde el usuario sube una foto de un plato y la IA desglosa los ingredientes probables y busca recetas similares.
- **Smart Pantry (Despensa Inteligente):** Un sistema que sugiere recetas basadas en lo que el usuario ya tiene en casa para reducir el desperdicio alimentario.
- **Realidad Aumentada (AR) en Menús:** Para los restaurantes asociados, permitir que los usuarios vean el plato en 3D sobre su mesa antes de pedirlo a través de la app.

3.2 Viabilidad técnica

Técnica: Es altamente viable. Las tecnologías de geolocalización (Google Maps API) y procesamiento de imágenes están maduras.

Económica: El coste inicial es elevado debido al almacenamiento de imágenes y video, pero se puede mitigar usando servicios de nube escalables (AWS/Azure) donde pagas por uso.

4.1 Requisitos técnicos

En cuanto a las copias de seguridad, este proyecto requiere que se haga una copia de seguridad de la base de datos cada semana, ya que, al tratarse de una red social, se mueven grandes cantidades de datos en poco tiempo y hay que salvar la mayor cantidad de datos recientes posibles y evitar que, en caso de fallo de la base de datos, no queden datos muy antiguos al restaurar la base de datos. Como nuestra base de datos está en la nube, podemos guardar una copia en la nube y otra descargarla a un disco duro local.

Para la escalabilidad, planeamos usar balanceadores de carga que nos proporciona AWS, así como duplicar la instancia principal del servidor web para disponer de los mismos datos para que más personas puedan usar la página sin que el rendimiento se vea afectado y no provoque caídas del servicio. Junto a esto, buscaremos mejorar el código del tema con IA para simplificarlo lo máximo posible para mejorar el rendimiento del sitio y que las máquinas no trabajen tanto al procesar tantas peticiones del usuario como lo que tiene que mostrar el sitio.

Nuestro servidor web permanecerá disponible todo el día y todos los días.

Como seguridad, implementamos un proxy para desviar el tráfico a los balanceadores de carga en caso de sobrecarga, sea por exceso de tráfico o por ataques DDoS. En los cuadros de búsqueda, se mejorará su código para filtrar lo que se introduce y evitar inyecciones SQL. En los login, se tendrá cuidado con la opción de "¿Has olvidado tu contraseña?" pidiendo el correo electrónico del usuario y evitar robos de cuenta. También, ajustaremos los permisos de acceso a distintas áreas de Wordpress, como "/wp-admin", para que el usuario normal sea incapaz de acceder a esta área redirigiéndolo a la página de inicio.

5. Diseño

a) Una Página Web, desarrollada con WordPress.

Desarrollo del Proceso de creación

WordPress es un sistema de gestión de contenidos desarrollado en PHP que corre sobre una base de datos MySQL o MariaDB, esta estructura es estándar en la mayoría de las instalaciones.

En nuestro contexto de proyecto tenemos la página Web de WordPress implementada en un servidor web altamente disponible y la base de datos de MariaDB en un Aurora RDS de AWS. WordPress utilizará Aurora RDS, como base de datos principal.

Tanto el WordPress como el Aurora RDS se despliegan en una subred privada, por lo que se solo será posible su acceso mediante el proxy inverso.

Nuestro sitio web, contará con un mínimo de 5 páginas y 6 entradas, se desglosan en la siguiente estructura:

Páginas

1. **Portada**: Será la página de inicio
2. **Quiénes somos**: Será una descripción sobre la empresa u organización que posee dicha página web: temática, historia, valores que promueve, equipo que está detrás de ella, misión y visión de la página
3. **Blog**: Contiene todas las entradas o publicaciones del sitio. WordPress se encargará automáticamente de ubicar los artículos. Tú le tienes que personalizar su diseño
4. **Contacto**: Formulario de contacto que se puede implementar con el Plugin Ninja Forms
5. **Política de privacidad**. Hace referencia a un documento que contiene los procedimientos y prácticas adoptados en una página, con el fin de brindarle

confianza al usuario con respecto al manejo de sus datos. Recuerda que esta página se genera automática desde el menú de Ajustes privacidad.

Entradas

1. Deben pertenecer a 2 o 3 categorías.
2. Deben tener una imagen y una imagen destacada.
3. Deben incluir un extracto

Para las entradas y las páginas utilizaremos Gutemberg, que es el editor por defecto de WordPress.

Tema o Plantilla

También tendremos que elegir y configurar un nuevo tema o plantilla para el WordPress, en nuestro caso aprovechando nuestra idea, utilizaremos el tema "Healthy Food Blogger" que nos aporta por defecto el diseño del sitio que queremos tener relacionado con la gastronomía.

Categorías y Etiquetas

En cuanto a las categorías, dentro de las entradas o publicaciones, deberemos de implementar dichas etiquetas o categorías para ordenar las entradas del blog de manera estética y eficaz.

Usuarios

Creación de al menos 6 usuarios para la administración y mantenimiento del sitio web.

Estos usuarios se encargarán de configurar partes menos significativas del sitio web, como:

1. Título del sitio
2. Descripción corta
3. Dirección de WordPress
4. Dirección del sitio
5. Dirección de correo electrónico

Creación de un Menú de Navegación

Deberemos de crear un menú de navegación situado en la cabecera del inicio del blog donde podamos acceder a todas las entradas del sitio web de manera eficaz.

Implementación de Plugins y Widgets

Los Plugins son carpetas de código que añaden e implementan funcionalidades a nuestro sitio web. Necesariamente tendremos que instalar varios de ellos para mejorar otras funciones o hacerlas más completas, entre ellos destacan:

1. Formulario de contacto con **Ninja Forms**.

2. **Ultimate Addons para Gutemberg**: Incorpora nuevos bloques a Gutemberg.
3. **EditorsKit**: nos da opciones para configurar los bloques.
4. **WooCommerce**: es un plugin gratuito para WordPress que convierte cualquier sitio web en una tienda en línea completa.

Uno de los **Plugins** mas importantes que tenemos que instalar es "**WP ALL IN ONE MIGRATION**", es un plugin que realizara la migración del sitio web a una EC2 del AWS configurada con alta disponibilidad como la primera, una parte fundamental de este proceso.

Los Widgets son bloques de información que se sitúan alrededor del contenido principal, estos serán usados para incluir un "Sidebar" donde se encuentren todos los widgets que implementemos.

Por último, tendremos que configurar funciones menos decisivas en el proceso de creación como los enlaces permanentes, ajustes de escritura, etc.

b) Una infraestructura desarrollada en AWS segura, escalable y altamente disponible.

El objetivo de nuestra idea es que sea desarrollada en un entorno seguro y disponible, para ello utilizaremos AWS Cloud.

AWS Cloud es una plataforma de computación en la nube de Amazon que permite usar servidores, almacenamiento y otros servicios a través de internet, sin necesidad de tener servidores físicos propios. En ella utilizaremos un Laboratorio donde se almacene toda la infraestructura, en cada laboratorio se establece un limite de dinero ficticio, todas las cosas que despleguemos gastan este dinero, por eso es muy importante saber gestionar bien el dinero del laboratorio, por lo que deberemos tener claro que necesitamos de AWS, para tener habilitado toda la infraestructura en cualquier momento.

VPC

Como inicio de la Infraestructura, tendremos la VPC, que es una red privada que creamos en AWS. Será la base de todo, a ella le asociaremos todo lo necesario para desplegar las bases de datos, servidores, proxys, instancias, etc.

Subredes

Al crear la VPC, se crean automáticamente al menos una subred pública y una subred privada, estas dos se encontrarán dentro de una zona de disponibilidad. Estas subredes son otra de las partes claves de la infraestructura, son necesarias para lograr los objetivos de accesos y seguridad.

Instancias

Las instancias son la parte mas importante de la infraestructura, tendremos dos dedicas a servidores web, necesitaremos una para el servidor web principal y otra que se usada para el balanceador de carga. Estas dos instancias estarán asociadas a la subred Privada, donde solo serán accesible gracias al proxy inverso.

Este proxy estará implantado en otra instancia, para controlar el acceso al servidor web y lograr esa alta seguridad que buscamos.

También contaremos con otra instancia que se encargara de monitorizar el estado de los servicios de los servidores web mediante un Script automatizado con Cron.

Tanto el proxy, como la instancia de monitorización serán asociadas a la subred pública.

Tablas de Rutas

Para cada subred existirá una tabla de rutas, donde se establecen las rutas de acceso.

Otras Funciones Asociadas a la VPC

1. **IGW**: El Internet Gateway es el componente que permite que una VPC tenga conexión directa con Internet.
2. **Gateway NAT**: permite que instancias en subredes privadas salgan a Internet, pero sin poder recibir conexiones desde fuera.
3. **Egress-Only Internet Gateway**: realiza la misma función que el Gateway NAT pero con la IPv6.
4. **Security Group**: son un firewall virtual para las instancias, en ellos se configuran restricciones de acceso, controlando el tráfico entrante y saliente.

Aurora RDS

Amazon Aurora RDS es un servicio de base de datos administrada de AWS, diseñado para ser más rápido, escalable y seguro que las bases de datos tradicionales.

Es el servicio que utilizaremos para almacenar nuestra base de datos de MariaDB, que utilizara el WordPress, en este servicio podremos establecer el lenguaje que queremos utilizar, nuestro caso seria el MySQL, ya que es el más efectivo.

c) Una Base de datos de MariaDB en Amazon RDS.

En nuestro proyecto, la base de datos se ha implementado utilizando MariaDB sobre el servicio Amazon RDS de AWS. Esta base de datos es la encargada de almacenar toda la información del sitio web desarrollado con WordPress.

MariaDB se ha elegido por su compatibilidad con MySQL, ya que WordPress funciona de forma nativa con este tipo de bases de datos, lo que facilita su integración y garantiza un buen rendimiento en aplicaciones web.

El servicio Amazon RDS nos permite disponer de una base de datos gestionada, lo que significa que AWS se encarga de tareas como copias de seguridad, actualizaciones y mantenimiento, mejorando la disponibilidad y reduciendo la carga de administración.

La base de datos se encuentra en una subred privada dentro de la VPC, por lo que no es accesible directamente desde Internet. Solo pueden conectarse a ella los servidores web donde está instalado WordPress, a través del puerto 3306 y mediante grupos de seguridad, garantizando así la seguridad de los datos.

WordPress utiliza esta base de datos para almacenar toda la información del sitio, como usuarios, publicaciones, recetas y configuraciones. La conexión se realiza mediante el endpoint de RDS, configurado en el archivo wp-config.php.

Además, se realizan copias de seguridad periódicas mediante snapshots de RDS, lo que permite recuperar la base de datos en caso de fallo o realizar migraciones entre entornos.

d) Implementación de Ansible y Terraform.

Terraform

Utilizaremos la utilidad de Terraform para automatizar el despliegue de la infraestructura de AWS.

Al menos buscamos automatizar el proceso de despliegue de Instancias EC2, definir varios recursos básicos de red (Security Groups, red, claves de acceso, etc.)

Utilizaremos ficheros de configuración para evitar utilizar la creación manual de recursos desde la consola del AWS.

Ansible

Por otra parte, utilizaremos Ansible para automatizar la configuración de las Instancias EC2, como la actualización de paquetes, servicios, instalación de programas, lenguajes, etc.

Al menos necesitamos automatizar la instalación y actualización de paquetes y servicios necesarios.

También aplicaremos Ansible para realizar las configuraciones básicas sobre los servicios instalados.

Por último, deberemos ejecutar de forma repetida de manera automática los playbooks de Ansible.

Organización de Terraform y Ansible

Podremos combinar las funciones de Terraform con las de Ansible y lograr la mayor automatización posible sobre AWS y sobre el mantenimiento de las Instancias EC2.

Tendremos que añadir al menos una tarea de mantenimiento o de gestión automatizada.

Todos los códigos de los Scripts de Terraform y Ansible, deben de tener unja estructura de carpetas de forma ordenada, un código legible, además de incluir comentarios para explicar líneas de código.

Todos los Scripts realizados, se almacenarán en una carpeta dentro del repositorio donde gestionamos todo el contenido del proyecto, esto permitirá disponer de ellos en cualquier momento, tener un historial de nuestras acciones sobre ellos y tenerlos guardados de forma ordenada y segura.

Por último, todo tendrá que estar justificado y documentado técnicamente para demostrar funcionalidad.

e) Script de Monitorización de servicios automatizado con Cron.

Objetivo del Script

Para lograr un mayor control sobre los servicios del trabajo, tendremos que desarrollar un Script en Bash o Python a elección nuestra, con este Script queremos monitorizar el estado de los servicios web y del sistema en general de los servidores web del proyecto.

Para llevarlo un paso más allá, combinaremos el Script con la función Cron, que nos permitirá programar la ejecución automática del Script en un periodo de tiempo, por ejemplo, podríamos tener un informe del estado de los servidore web cada hora para localizar y estar al tanto del estado de estos.

El Script guardará un registro en un log, donde podremos ver el contenido de la ejecución del script tanto como la fecha y hora de ejecución de este.

Modos de Ejecución

El Script deberá tener dos tipos de modos de ejecución:

Manual: de forma que exista un menú de interacción en caso de ejecutar el Script de forma interactiva, donde podamos ver varias opciones, como, arranque, parada, monitorización, estado, etc.

Automática: como hemos nombrado anteriormente utilizaremos Cron para programar la ejecución automática del Script, dejando evidencias de la ejecución en un Log.

Monitorización

El Script deberá Monitorizar varios elementos de los Servicios Web:

1. Estado del servicio
2. Conectividad local: puerto de escucha.
3. Respuesta HTTP: código de estado y tiempo de respuesta.
4. Recursos del sistema:
    - CPU
    - RAM
    - Disco Duro
5. Resultado final por ejecución: definir mensajes informativos como, Crit, Warning, etc.

Evidencias y Salidas

El log generado por el Script guardara evidencias de la ejecución de este con el Cron:

Fecha y Hora: del momento de la ejecución del Script.

Hostname o IP

Resultado de cada Checkeo

f) Configuración e Implantación de un Proxy Inverso.

En nuestra infraestructura se ha implementado un proxy inverso utilizando Nginx en una instancia situada en la subred pública. Este proxy actúa como punto de entrada a la aplicación, recibiendo todas las peticiones de los usuarios y redirigiéndolas hacia los servidores web que se encuentran en la subred privada.

El uso de un proxy inverso permite ocultar la estructura interna de la red, evitando que los servidores web y la base de datos estén expuestos directamente a Internet. De esta forma, solo la instancia del proxy es accesible públicamente, mejorando la seguridad del sistema.

Además, el proxy también realiza funciones de balanceo de carga, distribuyendo las peticiones entre los dos servidores web disponibles. Esto permite:

- Repartir el tráfico entre varias instancias
- Mejorar el rendimiento del sistema
- Evitar sobrecargas en un único servidor
- Aumentar la disponibilidad del servicio

En caso de que uno de los servidores falle, el proxy puede seguir redirigiendo el tráfico al resto de instancias disponibles, garantizando la continuidad del servicio.

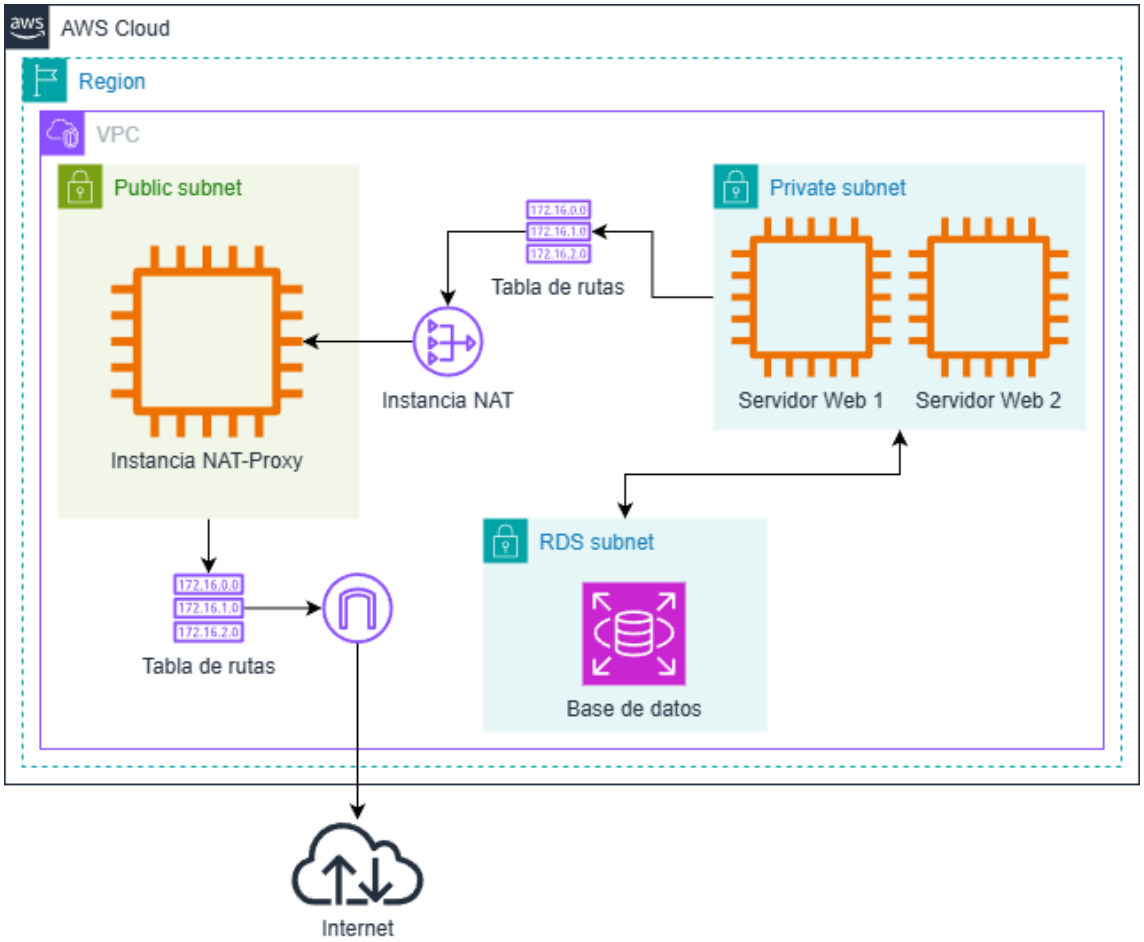
El proxy está configurado para reenviar las peticiones HTTP hacia los servidores donde se ejecuta WordPress, manteniendo las cabeceras necesarias para conservar la información del cliente, como la dirección IP original.

En conjunto, el proxy inverso no solo mejora la seguridad y organización de la red, sino que también permite escalar la aplicación y soportar un mayor número de usuarios de forma eficiente.

g) Diagrama de Infraestructura AWS

En AWS, la estructura que vamos a desplegar será una VPC con 3 subredes (Pública y Privada), en la subred publica habrá una Instancia EC2 que actuará como puerta de enlace NAT para ahorrar costes en el laboratorio; Esta instancia también será utilizada como proxy para tener balanceo de carga y dividir el tráfico de red en la web. En la instancia privada tendremos 2 Instancias EC2 que se utilizarán como servidores web para alojar nuestro sitio web; El sitio web estará echo mediante WordPress. La última subred será la que contenga el Amazon RDS (La base de datos) que será configurado mediante PHPMyAdmin.

En la siguiente imagen se muestra el diagrama logico de la implantación en AWS:



6. Planificación

La planificación del proyecto se ha realizado tomando como referencia directa el diseño de la infraestructura y de los servicios definidos. Para garantizar un despliegue correcto, se ha organizado la implementación en distintas fases que siguen el orden lógico de construcción del sistema, desde la red hasta la aplicación final.

Infraestructura de AWS

VPC

La implementación comienza con la creación de la VPC, que será la base de toda la infraestructura. En esta fase se definirá el rango de direcciones IP y se configurará el entorno de red donde se desplegarán todos los recursos.

Se comprobará que la VPC permite la comunicación interna entre los distintos servicios y que está correctamente asociada a los componentes necesarios.

Subredes

Una vez creada la VPC, se configurarán las subredes necesarias:

- Subred pública (para proxy y monitorización)
- Subred privada (para servidores web)
- Subred para la base de datos

Se verificará que cada subred está correctamente configurada según su función, asegurando que los recursos internos no sean accesibles desde Internet.

Tablas de rutas

Se crearán y asociarán las tablas de rutas correspondientes a cada subred:

- Subred pública con salida a Internet mediante Internet Gateway
- Subred privada con salida a Internet mediante NAT
- Subred de base de datos sin acceso directo a Internet

Se comprobará la conectividad entre subredes y la correcta configuración de las rutas.

Internet Gateway y NAT

Se implementarán los componentes necesarios para la conectividad:

- Internet Gateway para permitir acceso desde la subred pública
- Instancia NAT para permitir la salida a Internet desde la subred privada

Se configurará la instancia NAT con reenvío de paquetes y reglas de iptables, asegurando que las instancias privadas puedan acceder a Internet sin ser accesibles desde el exterior.

Grupos de seguridad

Se definirán los grupos de seguridad para cada recurso:

- Proxy: acceso HTTP/HTTPS desde Internet
- Servidores web: acceso solo desde el proxy
- Base de datos: acceso solo desde los servidores web

Se validará que únicamente el tráfico necesario esté permitido, garantizando la seguridad de la infraestructura.

Instancias EC2

Se desplegarán las distintas instancias necesarias para el sistema:

- Instancia Proxy/NAT en la subred pública
- Dos instancias de servidores web en la subred privada
- Instancia de monitorización en la subred pública

Se configurará el acceso mediante SSH y se verificará la conectividad entre instancias antes de continuar con la instalación de servicios.

Base de datos en Amazon RDS

Se creará la base de datos utilizando Amazon RDS con MariaDB.

Las tareas incluyen:

- Creación de la instancia RDS
- Configuración del grupo de subredes
- Definición de grupos de seguridad
- Creación de base de datos y usuarios

Se comprobará que la base de datos solo es accesible desde los servidores web y que el endpoint funciona correctamente.

Proxy inverso y balanceo de carga

Se configurará el proxy inverso en la instancia pública utilizando Nginx.

Las tareas serán:

- Instalación de Nginx
- Configuración del proxy inverso
- Configuración del balanceo de carga entre servidores web

Se verificará que el acceso desde Internet se realiza correctamente a través del proxy y que el tráfico se distribuye entre los servidores.

Servidores web y WordPress

En las instancias de la subred privada se realizará la instalación del entorno web:

- Instalación de Nginx, PHP y dependencias
- Instalación de WordPress
- Configuración del servidor web

Se comprobará que el sitio web funciona correctamente en cada servidor antes de integrarlo con el proxy.

Configuración de la base de datos en WordPress

Se configurará WordPress para conectarse a la base de datos RDS:

- Modificación del archivo wp-config.php
- Configuración del endpoint de RDS
- Pruebas de conexión

Se verificará que WordPress puede almacenar y recuperar datos correctamente.

PHPMyAdmin

Se implementará PHPMyAdmin para la gestión de la base de datos:

- Instalación del servicio
- Configuración del acceso al RDS
- Creación de usuarios

Se comprobará el acceso mediante navegador y la gestión de la base de datos.

Automatización con Terraform y Ansible

Terraform

Se utilizará Terraform para automatizar el despliegue de la infraestructura:

- Creación de recursos AWS
- Configuración de red y seguridad
- Despliegue de instancias

Se validará el funcionamiento de los scripts y la correcta creación de recursos.

Ansible

Se utilizará Ansible para la configuración de las instancias:

- Instalación de paquetes
- Configuración de servicios
- Automatización de tareas

Se ejecutarán playbooks para garantizar configuraciones homogéneas.

Monitorización

Se desarrollará un script de monitorización en Bash o Python:

- Comprobación del estado de servicios
- Monitorización de recursos (CPU, RAM, disco)
- Registro de resultados en logs

Se configurará su ejecución automática mediante Cron y se verificará el correcto funcionamiento de los registros.

Migración

Se planificará la migración de la infraestructura en caso necesario:

- Creación de imágenes AMI
- Despliegue en nuevo entorno
- Restauración de servicios

Se comprobará la integridad del sistema tras la migración.

Pruebas finales

Finalmente, se realizarán pruebas completas del sistema:

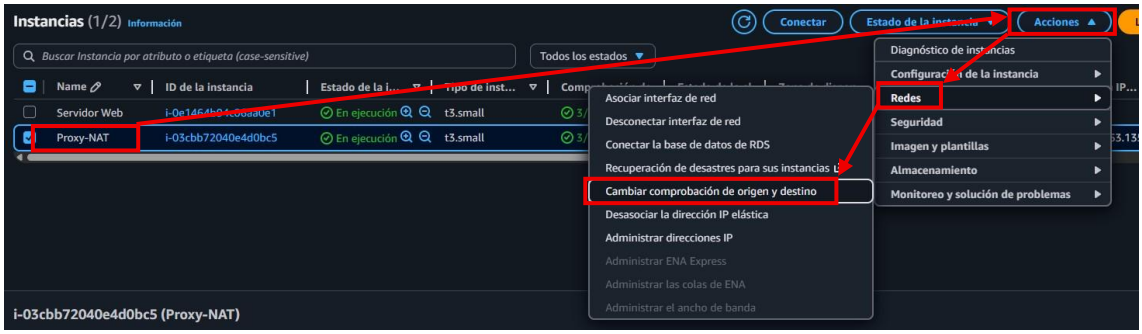
- Acceso desde Internet
- Funcionamiento del proxy
- Balanceo de carga
- Conectividad con la base de datos
- Funcionamiento de WordPress

Se validará que todo el sistema cumple con los requisitos de seguridad, disponibilidad y funcionamiento.

7. Implementación

Instancia NAT:

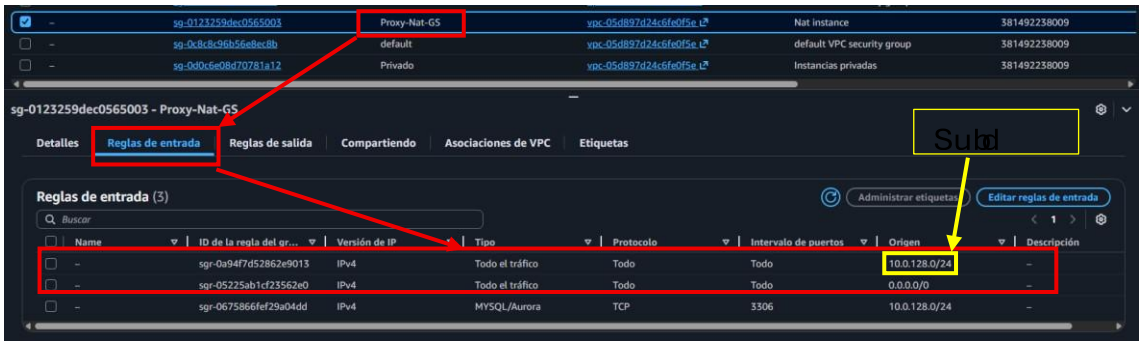
Desactivar la comprobación de origen y destino



Grupos de Seguridad:

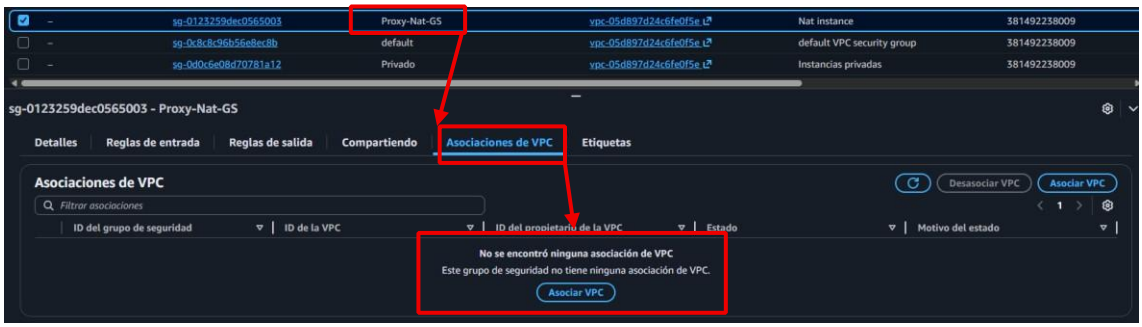
Grupo de Seguridad de la Instancia NAT:

Reglas de entrada para el grupo de seguridad del Bastión (Instancia NAT)



Reglas de salida para el grupo de seguridad del Bastión (Instancia NAT)

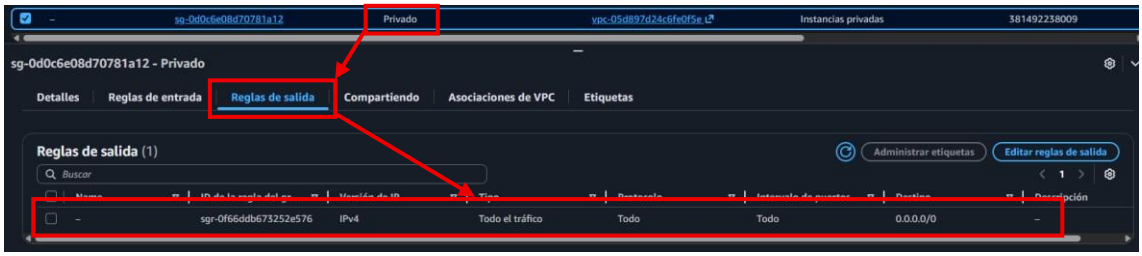
En asociaciones de VPC no debe haber nada



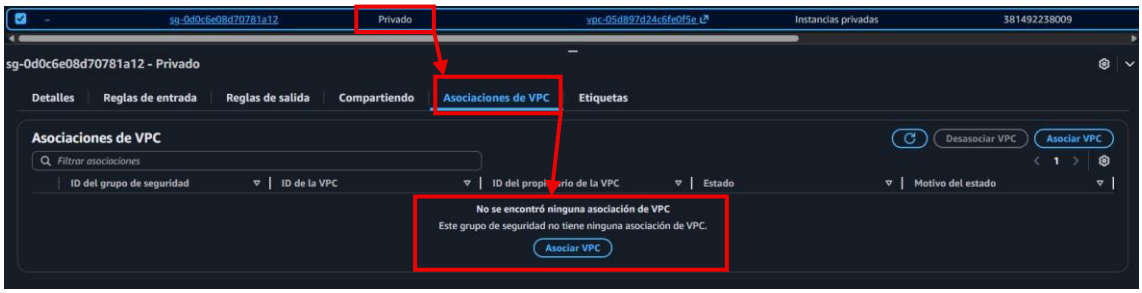
Gr u po de Seguri dad Privado:

Reglas de entrada del grupo de seguridad de la subred privada (Servidores Web)

Reglas de salida del grupo de seguridad de la subred privada (Servidores Web)

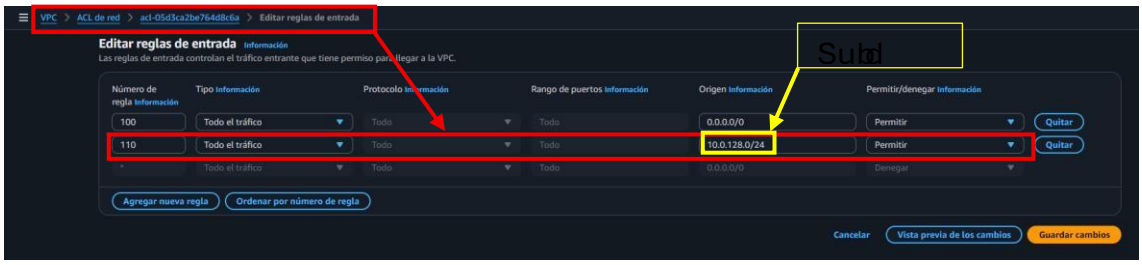


En asociaciones de VPC no debe haber nada

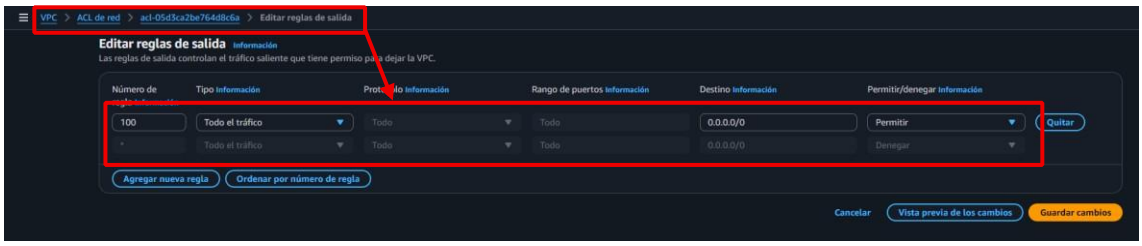


ACL de Red:

En el ACL de red de la vpc creada se debe añadir la regla 110 en reglas de entrada



En las reglas de salida no hay que añadir nada



Comandos en la Instancia NAT:

Actualizar repositorios de la instancia **sudo apt update** **sudo apt upgrade**

Comprobar el reenvío de paquetes IPv4 entre interfaces de red **sysctl net.ipv4.ip_forward**

Habilita el reenvío de paquetes IPv4 en el sistema, permitiendo que actúe como router entre redes

sudo sysctl -w net.ipv4.ip_forward=1

Abre el archivo /etc/sysctl.conf para descomentar la línea net.ipv4.ip_forward=1 **sudo nano /etc/sysctl.conf**

```
#net.ipv4.ip_forward=1
```

```
net.ipv4.ip_forward=1
```

Instala iptables-persistent, que permite guardar y restaurar automáticamente las reglas de iptables al iniciar el sistema **sudo apt install iptables-persistent**

Guarda las reglas actuales de iptables/netfilter para que se restauren automáticamente al reiniciar el sistema.

sudo netfilter-persistent save

Muestra de forma detallada las reglas de la tabla NAT de iptables, ahí debe aparecer la tarjeta de red del equipo

sudo iptables -t nat -L -n -v

```
ubuntu@ip-10-0-0-138:~$ sudo iptables -t nat -L -n -v
Chain PREROUTING (policy ACCEPT 804 packets, 48744 bytes)
 pkts bytes target     prot opt in     out     source               destination

Chain INPUT (policy ACCEPT 585 packets, 30268 bytes)
 pkts bytes target     prot opt in     out     source               destination

Chain OUTPUT (policy ACCEPT 468 packets, 34462 bytes)
 pkts bytes target     prot opt in     out     source               destination

Chain POSTROUTING (policy ACCEPT 17 packets, 1362 bytes)
 pkts bytes target     prot opt in     out     source               destination
  665 49220 MASQUERADE  0    --  *      ens5    0.0.0.0/0            0.0.0.0/0
```

Si no aparece, comprobar la tarjeta de red **ip a**

```
ubuntu@ip-10-0-0-138:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: ens5: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 9001 qdisc mq state UP group default qlen 1000
    link/ether 0a:ff:e1:cf:86:ed brd ff:ff:ff:ff:ff:ff
    inet 10.0.0.138/24 metric 100 brd 10.0.0.255 scope global dynamic ens5
       valid_lft 3402sec preferred_lft 3402sec
    inet6 fe80::8ff:e1ff:fecf:86ed/64 scope link
       valid_lft forever preferred_lft forever
```

Luego introducir este comando cambiando ens5 por el ID de la tarjeta **sudo iptables -t nat -A POSTROUTING -o ens5 -j MASQUERADE**

Añadir permanentemente la opción net.ipv4.ip_forward=1 a /etc/sysctl.conf para habilitar el reenvío IPv4 al arrancar el sistema.

echo "net.ipv4.ip_forward=1" | sudo tee -a /etc/sysctl.conf Muestra el valor actual de ip_forward en el kernel:

- 0 → deshabilitado (no se reenvían paquetes IPv4)
- 1 → habilitado (el sistema puede reenviar paquetes IPv4) **cat /proc/sys/net/ipv4/ip_forward**

Infraestructura de AWS

Configuración de la VPC

Esta es la configuración de nuestra VPC de AWS

Nombre	CIDR	Zona de Disponibilidad	Región
Proyectointermodular-vpc	10.0.0.0/16	us-east1	us-east-1a, us-east-1b (RDS)

Configuración de las subredes

Esta tabla muestra la configuración de las subredes de nuestra VPC

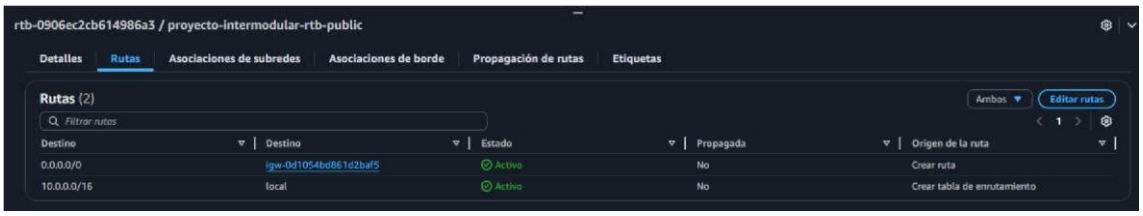
Subredes	CIDR	Zona de disponibilidad	Región
proyecto-intermodular-subnet-public1-useast-1a	10.0.0.0/24	us-east-1	us-east-1a
proyecto-intermodular-subnet-private1-useast-1a	10.0.128.0/24	us-east-1	us-east-1a
Subred-rds	10.0.30.0/24	us-east-1	us-east-1b

La subred del RDS forma parte de un grupo de subredes necesario para la creación de la base de datos, ya que requiere que se use otra región "de soporte"

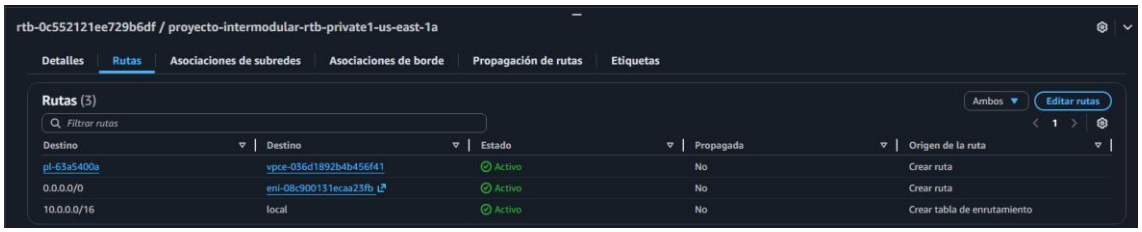
Configuración de las tablas de enrutamiento

Estas imágenes muestran las tablas de enrutamiento de las subredes

Subred pública



Subred privada



Configuración de las instancias

Esta tabla muestra las instancias que usamos en AWS

VPC	Instancias	Dirección IP	Subredes	Claves
Proyecto intermodular vpc	Proxy-NAT	54.211.153.135	proyecto-intermodular-subnet-public1-us-east1a	vockey
	Servidor Web	No recibe IP pública	proyecto-intermodular-subnet-private1-us-east1a	vockey
	Servidor Web 2	No recibe IP pública	proyecto-intermodular-subnet-private1-us-east1a	vockey
	Monitorización	34.233.2.104	proyecto-intermodular-subnet-public1-us-east1a	vockey

Descripción

Proxy-NAT: Servidor Proxy encargado de recibir y redirigir el tráfico a los servidores web, que será accesible para todos

Servidor Web: Servidor web que almacena Wordpress

Servidor Web 2: Servidor de balanceo de carga para el servidor web

Monitorización: Instancia utilizada que actúa como máquina local para monitorizar los servicios del proxy y el servidor web

Grupos de seguridad

Estos son los grupos de seguridad de las instancias y de la base de datos

Grupos de seguridad	Reglas de entrada	Reglas de salida	Instancias asignadas
Proxy-NAT-GS	10.0.128.0/24, 0.0.0.0/0, 10.0.128.0/24:3306	0.0.0.0/0 (Todo el tráfico)	Proxy-NAT
ec2-rds-1	10.0.128.0/24:3306	rds-ec2-1 (Grupo de seguridad)	FATGRAM RDS
rds-ec2-1	ec2-rds-1	Sin regla de salida	FATGRAM RDS
Privado	10.0.128.0/24:3306	0.0.0.0/0	Servidor Web 1 y 2

Descripción

Proxy-NAT-GS	Grupo de seguridad de la instancia NAT
ec2-rds-1	Grupo de seguridad para el RDS-Instancia
rds-ec2-1	Grupo de seguridad para Instancia-RDS
Privado	Grupo de seguridad para las instancias privadas

Configuración de la base de datos RDS

Esta es la configuración de la base de datos donde se almacenan los datos del servidor

Name	ID de grupo de seguridad	Nombre del grupo de seguridad	Descripción	Número de reglas de entrada	Número de reglas de salida
-	sg-0123259dec0565003	Proxy-Nat-GS	Nat instance	3 Entradas de permisos	1 Entrada de permiso
-	sg-0c8c8c96b56e8ec8b	default	default VPC security group	1 Entrada de permiso	1 Entrada de permiso
-	sg-06422ba9de78f7d57	ec2-rds-1	Security group attached to instances to ...	1 Entrada de permiso	1 Entrada de permiso
-	sg-07c8e64d7fc21f69a	rds-ec2-1	Security group attached to fatgram to a...	1 Entrada de permiso	0 Entradas de permisos
-	sg-0d0c6e08d70781a12	Privado	Instancias privadas	3 Entradas de permisos	1 Entrada de permiso

fatgram — Modificar — Acciones

Resumen

Identificador de base de datos	Estado	Rol	Motor	Recomendaciones
fatgram	Disponible	Instancia	MariaDB	2 Informativo
CPU	**Clase**	**Actividad actual**	**Región y AZ**	
4.24%	db.t4g.micro	0 Conexiones	us-east-1a	

Conectividad y seguridad | Supervisión | Registros y eventos | Configuración | Mantenimiento y copias de seguridad | Migraciones de datos | Etiquetas | Recomendaciones

Instancia

Configuración
- ID de instancia de base de datos: fatgram
- Versión del motor: 11.4.8
- Nombre de base de datos: -
- License model: General Public License
- Grupos de opciones: default:mariadb-11-4 (En sincronización)
- Nombre de recurso de Amazon (ARN)

Clase de instancia
- Clase de instancia: db.t4g.micro
- vCPU: 2
- RAM: 1 GB

Disponibilidad
- Nombre de usuario maestro: root
- Contraseña maestra

Almacenamiento principal
- Cifrado: Habilitado
- Clave de AWS KMS: aws/rds
- Tipo de almacenamiento: SSD de uso general (gp2)
- Almacenamiento: 20 GiB
- IOPS provisionadas: -
- Rendimiento de almacenamiento

Supervisión
- Tipo de supervisión: Database Insights: estándar
- Información sobre rendimiento: Desactivado
- Monitorización mejorada: Desactivado

Para la creación de la base de datos, se ha tenido que crear un grupo de subredes que junta las subredes propias de la VPC y otra subred que pertenezca a otra región para que sirva de soporte en caso de que la primera región falle.

rds fatgram

Detalles del grupo de subredes

- ID de VPC: vpc-05d897d24c6fe0f5e
- ARN: arn:aws:rds:us-east-1:381492238009:subgrp:rds fatgram
- Tipos de red compatibles: IPv4
- Descripción: grupo para base de datos

Subredes (3)

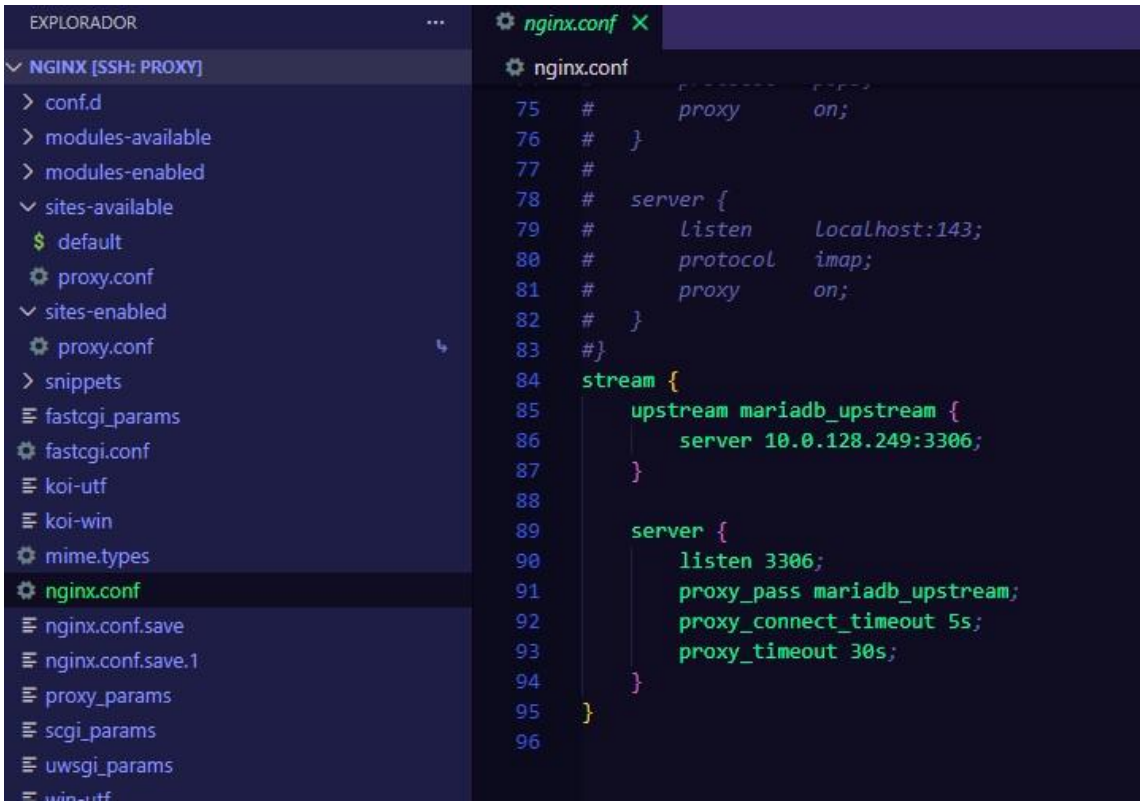
Zona de disponibilidad	Nombre de subred	ID de subred	Bloque de CIDR
us-east-1a	proyecto-intermodular-subnet-private1-us-east-1a	subnet-03d1e360c40a0c150	10.0.128.0/24
us-east-1b	subred-rds	subnet-0cc4e94d9455e69f6	10.0.30.0/28
us-east-1a	proyecto-intermodular-subnet-public1-us-east-1a	subnet-038317d64bba769dd	10.0.0.0/24

Proxy en Instancia NAT de AWS:

Para este proyecto, en AWS tenemos una VPC con 2 subredes, la privada y la pública, en la publica esta la Instancia NAT configurada anteriormente, y en la privada el servidor web; Como desde internet no se puede acceder a las instancias de la subred privada, para poder implementar y acceder a nuestro sitio web debemos implementar un Proxy que actúe de puente entre el cliente y la web, para ello configuramos un proxy en el servidor web nginx.

Configuración en Nginx:

1. Archivo nginx.conf:



- Bloque upstream: Aquí se define el destino, creando un grupo llamado mariadb_upstream que apunta a la IP privada del servidor web

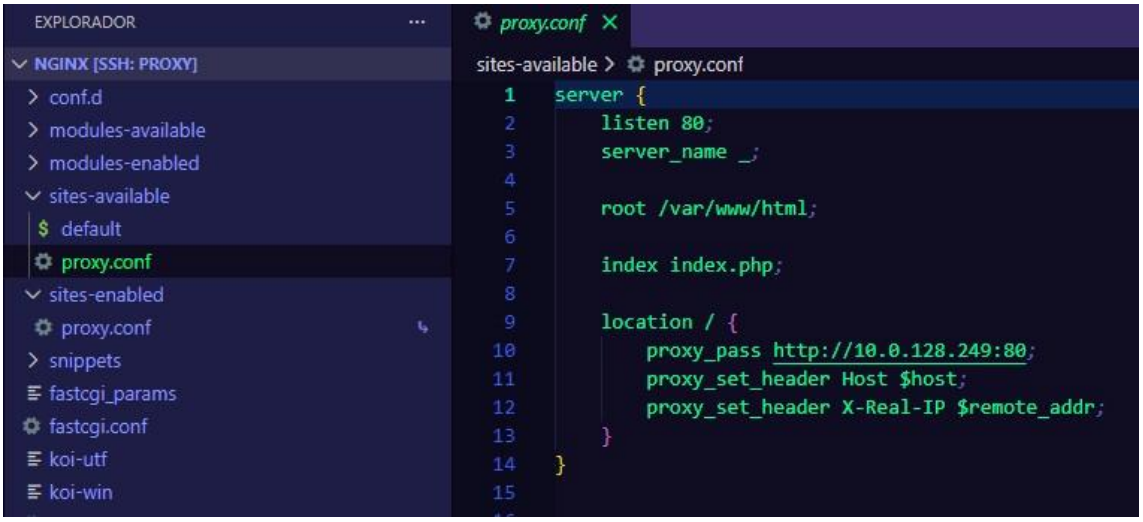
(10.0.128.249) en el puerto estándar de MariaDB/MySQL (3306) para poder acceder al RDS.

- Bloque server: Le dice a Nginx el puerto que tiene que escuchar.
- proxy_pass: Redirige cualquier conexión que llegue a ese puerto directamente hacia el upstream de arriba

- Timeouts: Aquí he configurado un tiempo de espera de conexión de 5 segundos y un tiempo de inactividad de 30 segundos, para no dejar conexiones colgadas si la base de datos no responde.

2. Archivo proxy.conf:

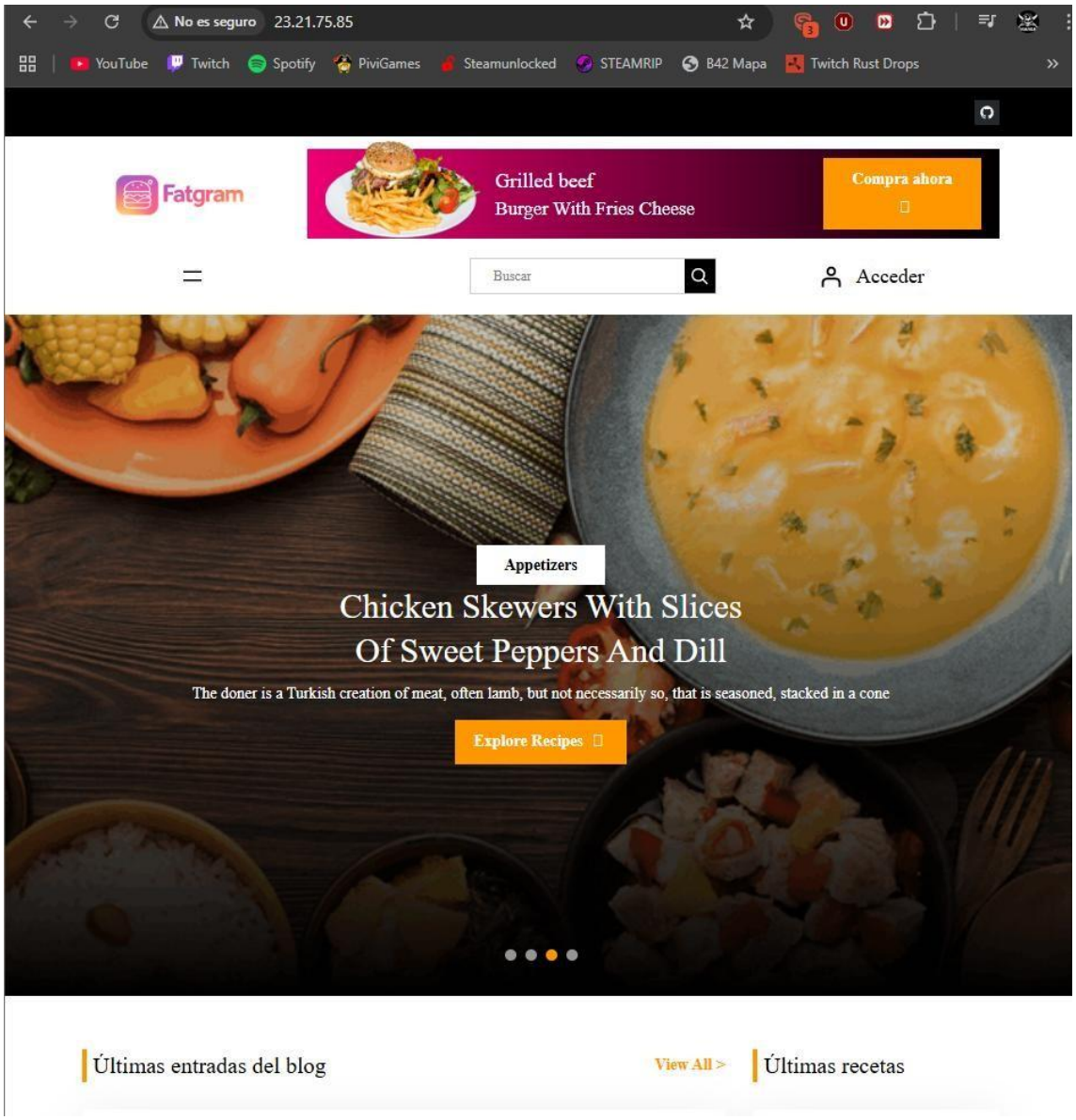
En lugar de configurar un sitio web normal y corriente, se configurará de esta manera para que el "sitio" actúe como proxy:



- listen 80: El proxy está atento a cualquier petición web que llegue por el puerto 80
- server_name _: Indica que este bloque de servidor responderá a cualquier dominio o dirección IP que apunte a esta máquina.
- location / { ... }: Define que todas las peticiones (la raíz / y todo lo que cuelgue de ella) deben ser enviadas al backend.
- proxy_pass http://10.0.128.249:80: Aquí es donde Nginx actúa como puente. Recibe la petición del usuario y la reenvía al servidor web privado por HTTP.
- Cabeceras (proxy_set_header):
    - Host $host: Pasa el nombre del sitio original al servidor backend.
    - X-Real-IP $remote_addr: Le dice al backend cuál es la IP real del usuario, para que en los logs del backend no aparezca siempre la IP del proxy.

Resultado:

Al acceder a la IP pública del proxy, se nos redirige al servidor web en la instancia privada, donde está alojado el sitio web real


No es seguro
23.21.75.85
YouTube
Twitch
Spotify
PiviGames
Steamunlocked
STEAMRIP
B42 Mapa
Twitch Rust Drops
Fatgram
Grilled beef
Burger With Fries Cheese
Compra ahora
Buscar
Acceder
Appetizers
Chicken Skewers With Slices
Of Sweet Peppers And Dill
The doner is a Turkish creation of meat, often lamb, but not necessarily so, that is seasoned, stacked in a cone
Explore Recipes
Últimas entradas del blog
View All >
Últimas recetas

WordPress:

Para crear nuestro sitio web, en este proyecto debemos hacerlo utilizando WordPress, para ello, debemos instalarlo de la siguiente manera:

Instalar nginx:

Instalar servidor web nginx para configurar el acceso web

```
ubuntu@ip-10-0-128-249:~$ sudo apt install nginx
```

Instalar php y mysql:

Instalar estas herramientas para su posterior configuración y funcionamiento

```
ubuntu@ip-10-0-128-249:/etc/nginx/sites-available$ apt install php-fpm php-mysql
```

Crear archivo de configuración en Nginx:

Crear el archivo de configuración del sitio web del WordPress

```
ubuntu@ip-10-0-128-249:~$ cd /etc/nginx/sites-available/
```

Crear una copia del archivo por defecto

```
ubuntu@ip-10-0-128-249:/etc/nginx/sites-available$ cp default wordpress
```

Modificar el archivo de configuración:

```
ubuntu@ip-10-0-128-249:/etc/nginx/sites-available$ sudo nano wordpress
```

```
  GNU nano 7.2                          wordpress
# Default server configuration
#
server {
        listen 80 default_server;
        listen [::]:80 default_server;

        # SSL configuration
        #
        # listen 443 ssl default_server;
        # listen [::]:443 ssl default_server;
        #
        # Note: You should disable gzip for SSL traffic.
        # See: https://bugs.debian.org/773332
        #
        # Read up on ssl_ciphers to ensure a secure configuration.
        # See: https://bugs.debian.org/765782
        #
        # Self signed certs generated by the ssl-cert package
        # Don't use them in a production server!
        #
        # include snippets/snakeoil.conf;

        root /var/www/html;

        # Add index.php to the list if you are using PHP
        index index.php index.html index.htm index.nginx-debian.html;

        server_name _;

        location / {
                # First attempt to serve request as file, then
                # as directory, then fall back to displaying a 404.
                try_files $uri $uri/ /index.php?$args;
        }

        # pass PHP scripts to FastCGI server
        #
        location ~ \.php$ {
                include snippets/fastcgi-php.conf;

                # With php-fpm (or other unix sockets):
                fastcgi_pass unix:/run/php/php8.3-fpm.sock;
                # With php-cgi (or other tcp sockets):
                #fastcgi_pass 127.0.0.1:9000;
        }

        # deny access to .htaccess files, if Apache's document root
        # concurs with nginx's one
        #
        location ~ /\.ht {
                deny all;
```

1. Escucha de tráfico (Puerto 80)

```
server {
        listen 80 default_server;
        listen [::]:80 default_server;
```

Configura el servidor para escuchar peticiones en el puerto 80 default_server significa que este es el sitio que responderá si alguien entra a la IP del servidor sin un dominio específico configurado.

2. Directorio raíz y archivos índice

```
root /var/www/html;

# Add index.php to the list if you are using PHP
index index.php index.html index.htm index.nginx-debian.html;
```

root: Define que los archivos del WordPress están físicamente en la carpeta /var/www/html.

index: Le dice a Nginx que, si alguien entra a la raíz, busque primero el archivo index.php.

3. Manejo de URLs (Enlaces permanentes)

```
server_name _;

location / {
        # First attempt to serve request as file, then
        # as directory, then fall back to displaying a 404.
        try_files $uri $uri/ /index.php?$args;
}
```

Esta es la pieza clave para WordPress. Intenta buscar el archivo solicitado (como una imagen o un CSS).

Si el archivo no existe, envía la petición al index.php con todos los argumentos. Sin esto, se verían errores 404 al navegar por el sitio

4. Procesamiento de PHP (El motor)

```
location ~ \.php$ {
        include snippets/fastcgi-php.conf;

        # With php-fpm (or other unix sockets):
        fastcgi_pass unix:/run/php/php8.3-fpm.sock;
        # With php-cgi (or other tcp sockets):
        #fastcgi_pass 127.0.0.1:9000;
}
```

Nginx por sí solo no sabe leer PHP. Cuando ve un archivo .php, lo envía a un programa externo llamado PHP-FPM mediante un "socket" y PHP procesa la base de datos y genera el HTML que el usuario finalmente ve.

Instalar WordPress:

Para instala WordPress hay que descargar el archivo comprimido con todo lo necesario utilizando este comando:

```
ubuntu@ip-10-0-128-249:~$ sudo wget https://es.wordpress.org/latest-es_ES.zip
--2026-02-05 17:58:52--  https://es.wordpress.org/latest-es_ES.zip
Resolving es.wordpress.org (es.wordpress.org)... 198.143.164.252, 2607:f978:5:8002::c68f:a4fc
Connecting to es.wordpress.org (es.wordpress.org)|198.143.164.252|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 36112982 (34M) [application/zip]
Saving to: 'latest-es_ES.zip.1'

latest-es_ES.zip.1        100%[===================================>]  34.44M  96.9MB/s    in 0.4s

2026-02-05 17:58:53 (96.9 MB/s) - 'latest-es_ES.zip.1' saved [36112982/36112982]
```

Lo descomprimimos con este otro comando:

```
ubuntu@ip-10-0-128-249:~$ sudo unzip latest-es_ES.zip
```

```
ubuntu@ip-10-0-128-249:/var/www/html$ ls
index.php                          wordpress            wp-config.php      wp-login.php
info.php                           wp-activate.php      wp-content         wp-mail.php
license.txt                        wp-admin             wp-cron.php        wp-settings.php
phpMyAdmin-5.2.1-all-languages.zip wp-blog-header.php   wp-includes        wp-signup.php
phpmyadmin                         wp-comments-post.php wp-links-opml.php  wp-trackback.php
readme.html                        wp-config-sample.php wp-load.php        xmlrpc.php
ubuntu@ip-10-0-128-249:/var/www/html$
```

Configuramos este archivo:

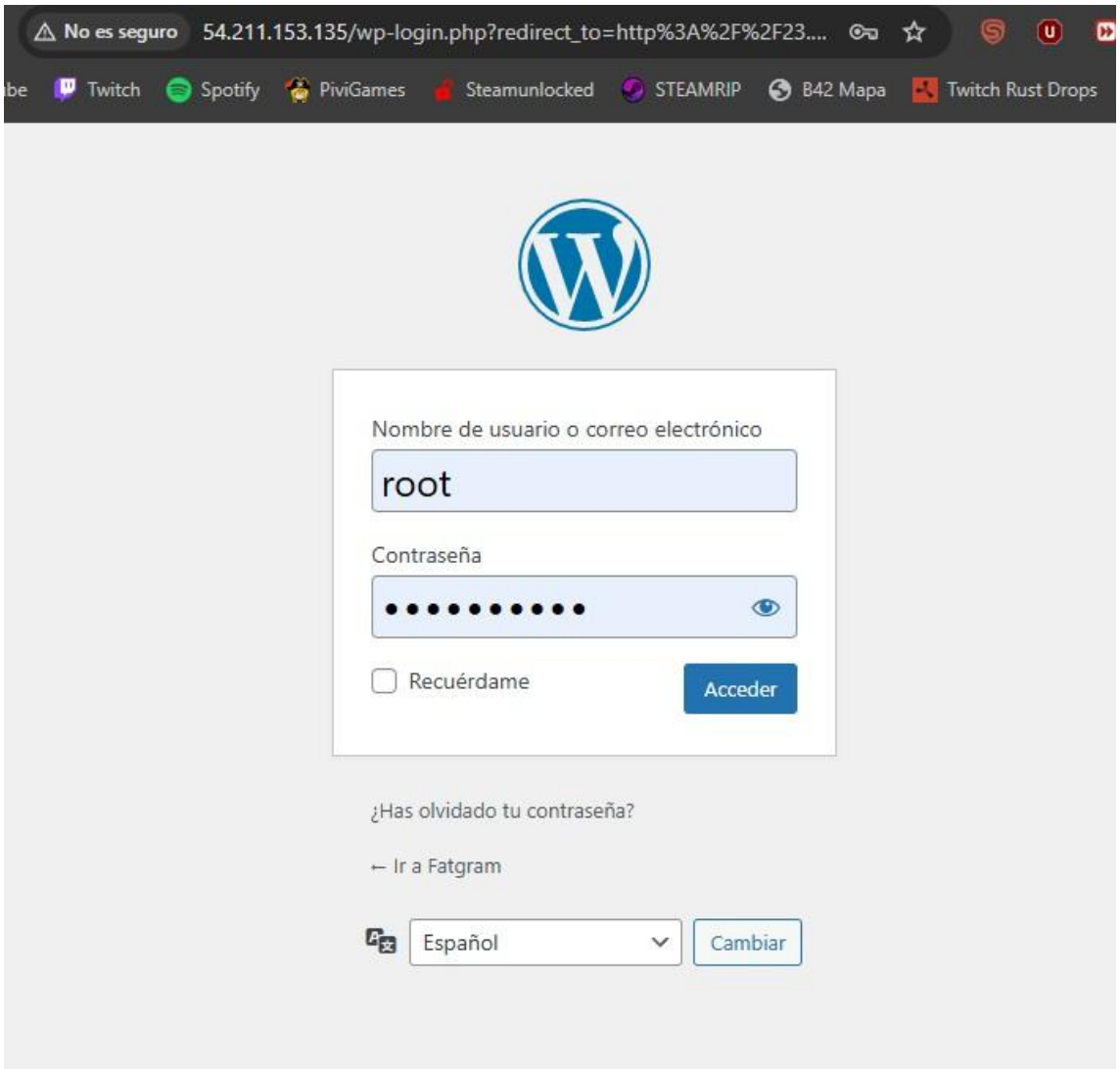
```
ubuntu@ip-10-0-128-249:/var/www/html$ sudo nano wp-config.php
```

En el apartado “DB_HOST” ponemos el EndPoint del Amazon RDS:

```
  GNU nano 7.2                                  wp-config.php
<?php
/**
 * The base configuration for WordPress
 *
 * The wp-config.php creation script uses this file during the installation.
 * You don't have to use the website, you can copy this file to "wp-config.php"
 * and fill in the values.
 *
 * This file contains the following configurations:
 *
 * * Database settings
 * * Secret keys
 * * Database table prefix
 * * ABSPATH
 *
 * @link https://developer.wordpress.org/advanced-administration/wordpress/wp-config/
 *
 * @package WordPress
 */

// ** Database settings - You can get this info from your web host ** //
/** The name of the database for WordPress */
define( 'DB_NAME', 'wordpress' );

/** Database username */
define( 'DB_USER', 'root' );

/** Database password */
define( 'DB_PASSWORD', 'usuario2' );

/** Database hostname */
define( 'DB_HOST', 'fatgram.cqg5ve3796m7.us-east-1.rds.amazonaws.com' );

/** Database charset to use in creating database tables. */
define( 'DB_CHARSET', 'utf8mb4' );

/** The database collate type. Don't change this if in doubt. */
define( 'DB_COLLATE', '' );
```

Resultado:

Si accedemos desde el navegador con la IP del proxy configurado anteriormente ya tendremos la interfaz web del WordPress


No es seguro
54.211.153.135/wp-login.php?redirect_to=http%3A%2F%2F23....
Twitch
Spotify
PiviGames
Steamunlocked
STEAMRIP
B42 Mapa
Twitch Rust Drops
Nombre de usuario o correo electrónico
root
Contraseña
Recuérdame
Acceder
¿Has olvidado tu contraseña?
← Ir a Fatgram
Español
Cambiar

PHPMyAdmin:

Descargar él .zip de PHPMyAdmin:

Con este comando se descargan todos los archivos necesarios en un comprimido:

```
ubuntu@ip-10-0-128-249:~$ sudo wget https://files.phpmyadmin.net/phpMyAdmin/5.2.1/phpMyAdmin-5.2.1-all-languages.zip
```

Lo descomprimimos:

```
ubuntu@ip-10-0-128-180:/var/www/html$ sudo unzip phpMyAdmin-5.2.1-all-languages.zip
```

```
ubuntu@ip-10-0-128-180:/var/www/html$ cd phpmyadmin/
ubuntu@ip-10-0-128-180:/var/www/html/phpmyadmin$ ls
CONTRIBUTING.md     babel.config.json      examples     locale                 sql        yarn.lock
ChangeLog           composer.json          favicon.ico  package.json           templates
LICENSE             composer.lock          index.php    robots.txt             themes
README              config.sample.inc.php  js           setup                  url.php
RELEASE-DATE-5.2.1  doc                    libraries    show_config_errors.php vendor
ubuntu@ip-10-0-128-180:/var/www/html/phpmyadmin$
```

Archivo de configuración de PHPMyAdmin:

En este archivo:

```
ubuntu@ip-10-0-128-249:/var/www/html/phpmyadmin$ sudo nano config.inc.php
```

En el apartado ‘Servers’ ponemos el EndPoint del Amazon RDS

```
  GNU nano 7.2                    config.inc.php
<?php
/**
 * phpMyAdmin sample configuration, you can use it as base for
 * manual configuration. For easier setup you can use setup/
 *
 * All directives are explained in documentation in the doc/ folder
 * or at <https://docs.phpmyadmin.net/>.
 */

declare(strict_types=1);

$i = 1;

/**
 * This is needed for cookie based authentication to encrypt the cookie.
 * Needs to be a 32-bytes long string of random bytes. See FAQ 2.10.
 */
$cfg['Servers'][$i]['blowfish_secret'] = '7h8j2kLp9mN3bV5cxZ1qA4sW6eD8rT0y'; /* Y

/**
 * Servers configuration
 */

/**
 * First server
 */
$i++;
/* Authentication type */
$cfg['Servers'][$i]['auth_type'] = 'cookie';
/* Server parameters */
$cfg['Servers'][$i]['host'] = 'fatgram.cqg5ve3796m7.us-east-1.rds.amazonaws.com';
$cfg['Servers'][$i]['port'] = '3306';
$cfg['Servers'][$i]['compress'] = false;
$cfg['Servers'][$i]['AllowNoPassword'] = false;
```

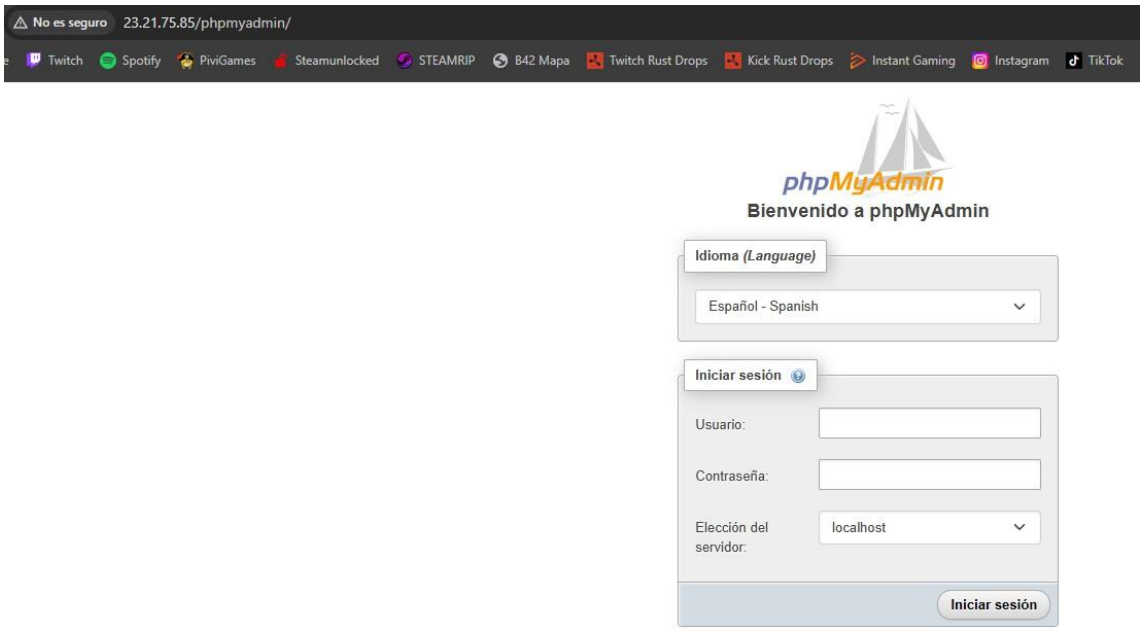
Crear usuario para PHPMyAdmin:

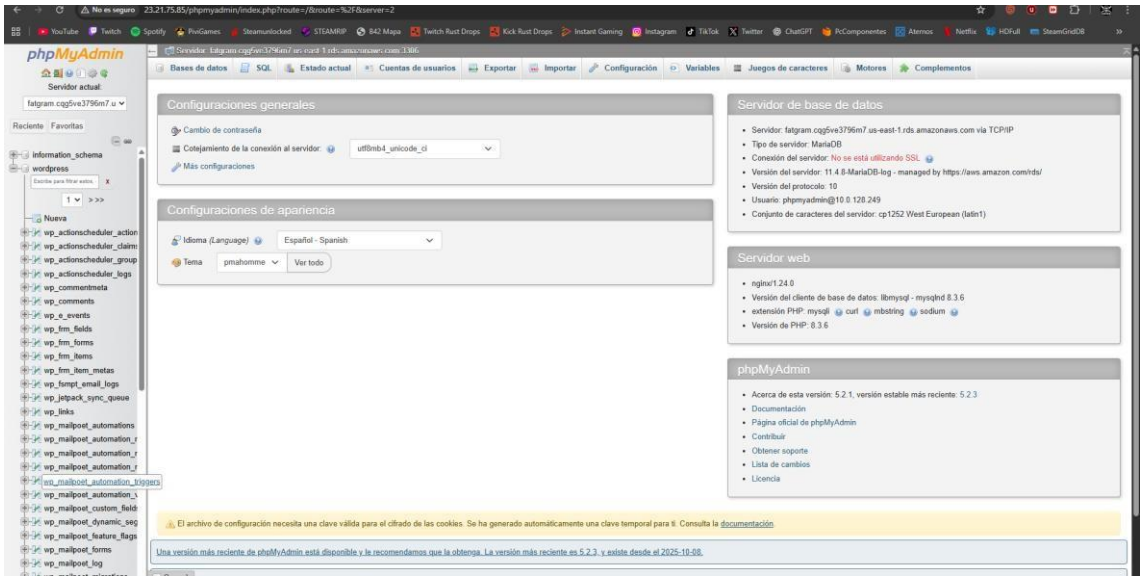
En MariaDB, crear un usuario con privilegios en la base de datos del WordPress para poder modificarla luego en entorno web:

```
ubuntu@ip-10-0-128-249:~$ mysql -h fatgram.cqg5ve3796m7.us-east-1.rds.amazonaws.com -P 3306 -u root -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 482
Server version: 11.4.8-MariaDB-log managed by https://aws.amazon.com/rds/

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> SELECT user, host FROM mysql.user;
+-------------+-----------------------------------------------+
| User        | Host                                          |
+-------------+-----------------------------------------------+
| phpmyadmin  | %                                             |
| root        | %                                             |
| wordpress   | fatgram.chgqawmakccd.us-east-1.rds.amazonaws.com |
| FATGRAM     | localhost                                     |
| mariadb.sys | localhost                                     |
| rdsadmin    | localhost                                     |
+-------------+-----------------------------------------------+
6 rows in set (0.001 sec)

MariaDB [(none)]> SHOW GRANTS FOR 'phpmyadmin'@'%';
+-----------------------------------------------------------------------------------------------------------+
| Grants for phpmyadmin@%                                                                                   |
+-----------------------------------------------------------------------------------------------------------+
| GRANT USAGE ON *.* TO `phpmyadmin`@`%` IDENTIFIED BY PASSWORD '*6F836E74EEA49AFB8061B6168FB422AB4FE60382' |
| GRANT ALL PRIVILEGES ON `wordpress`.* TO `phpmyadmin`@`%` WITH GRANT OPTION                               |
+-----------------------------------------------------------------------------------------------------------+
2 rows in set (0.002 sec)

MariaDB [(none)]>
```

Interfaz Web:

En el navegador, introducir la IP del proxy añadiendo /phpmyadmin y acceder con el usuario creado



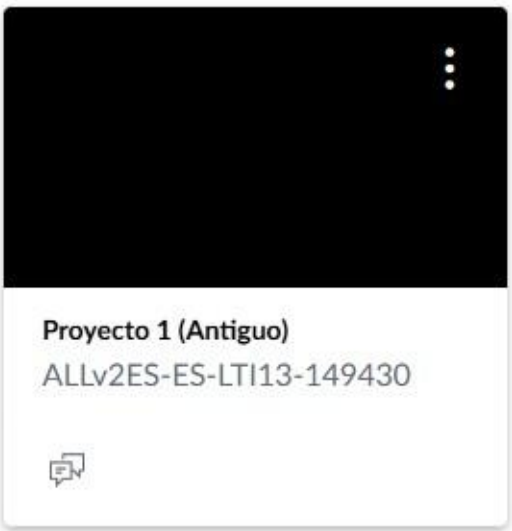


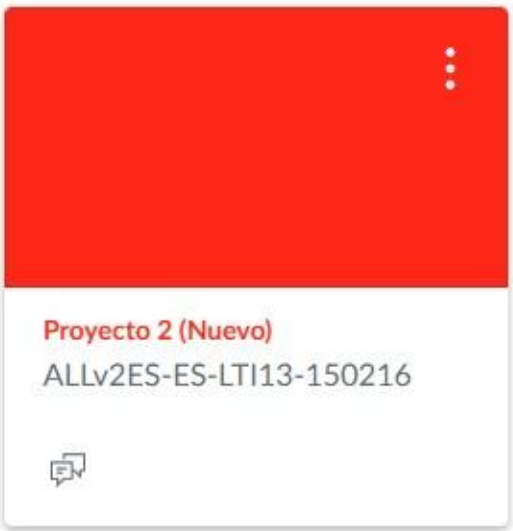
Migración:

Debido a las limitaciones de gasto de los laboratorios AWS (50$), hemos tenido que migrar todo nuestro proyecto a un nuevo laboratorio para evitar alcanzar el límite de gasto y perderlo todo; Así es como hemos migrado las Instancias:

Laboratorios:

Estos son los 2 laboratorios que nos van a hacer falta, el antiguo es el que ya tiene demasiado gasto, y el nuevo es al que vamos a trasladar todo:





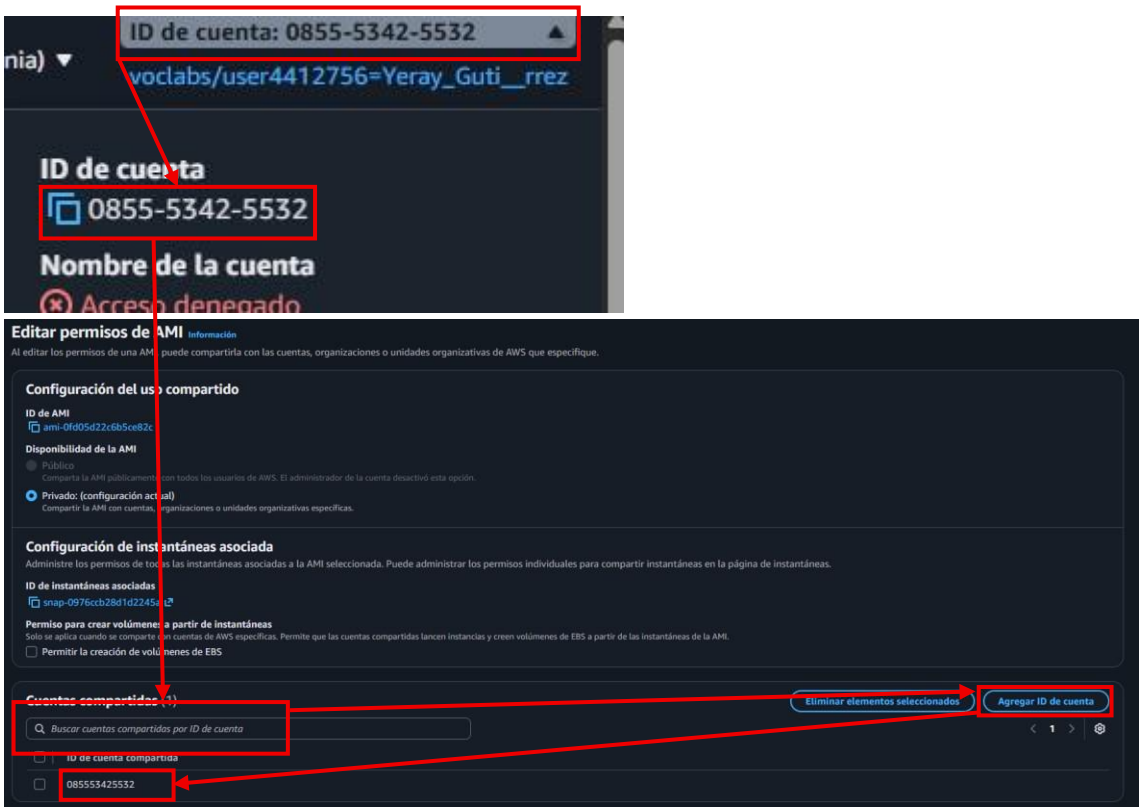
1. Crear AMIs:

Para la migración de las instancias, necesitaremos crear al menos una AMI de cada instancia, lo cual se hace en el menú de instancias seleccionar Acciones>Imagen y plantillas>Crear imagen. Esto se debe hacer con las 4.

2. Compartir Instancias:

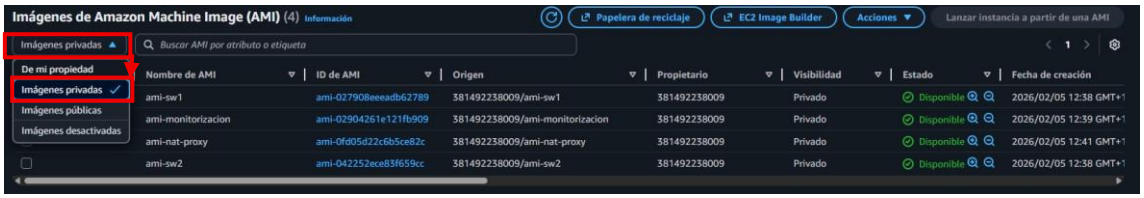
Una vez creadas las instancias, debemos ir al menú de AMIs, seleccionar la instancia e ir a Acciones>Editar permisos de AMI; Esto se debe hacer con todas.

Ahora debemos ir a la sección de cuentas compartidas y agregar el ID de la cuenta del nuevo laboratorio:

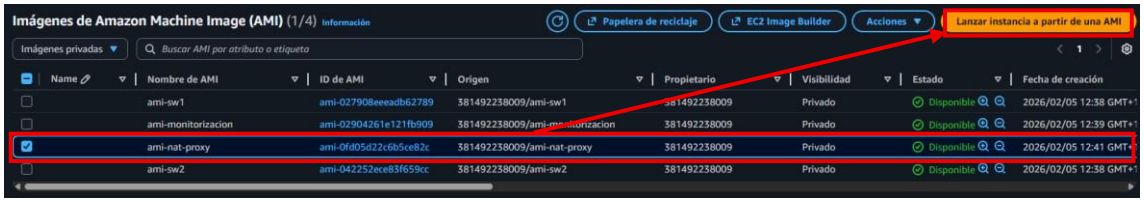


3. Lanzar las Instancias desde el nuevo laboratorio:

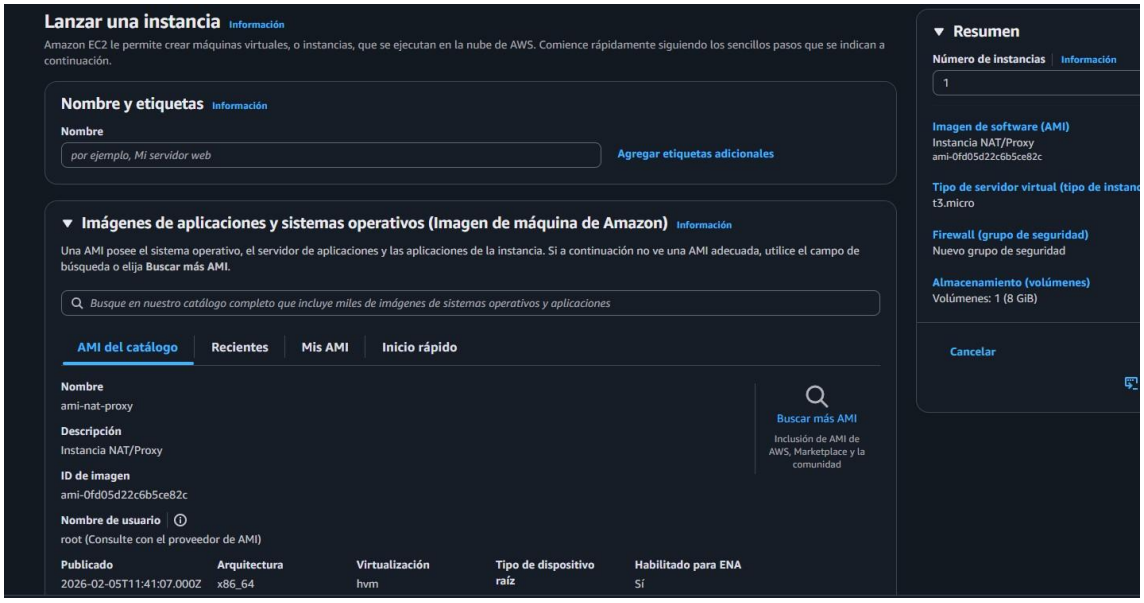
En el nuevo laboratorio, debemos ir al mismo menú de AMIs y donde por defecto pone “De mi propiedad” cambiar a “Imágenes privadas”:



Luego seleccionar la AMI e ir a “Lanzar instancia a partir de una AMI:



Ahora se lanza la Instancia como una normal, teniendo que seleccionar los grupos de seguridad, VPCs, etc.

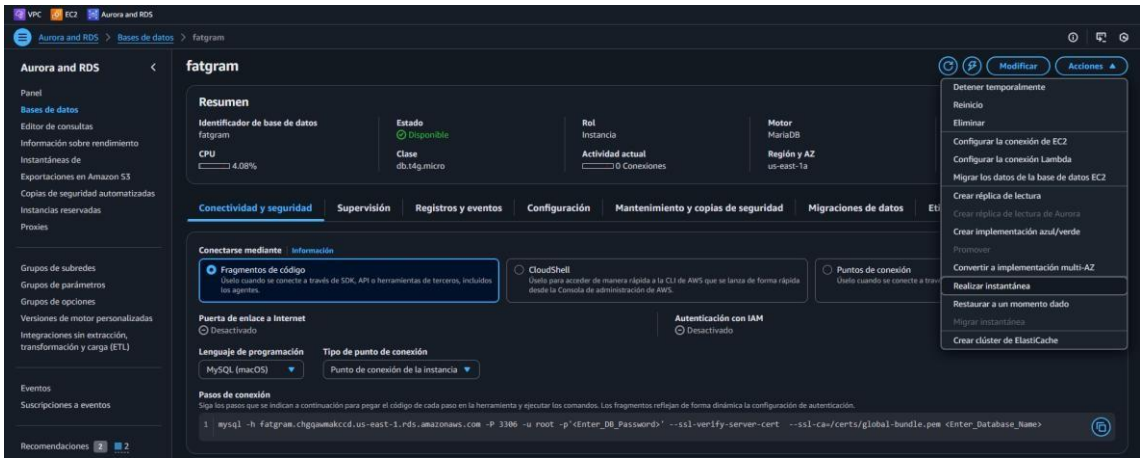


Resultado:

Después de repetir el proceso anterior con el resto de AMIs, ya tendremos todas las instancias migradas completamente funcionales y exactamente igual a como estaban en el laboratorio anterior:

Crear una instantánea del Amazon RDS:

Ir al RDS e ir a Acciones>Realizar instantánea

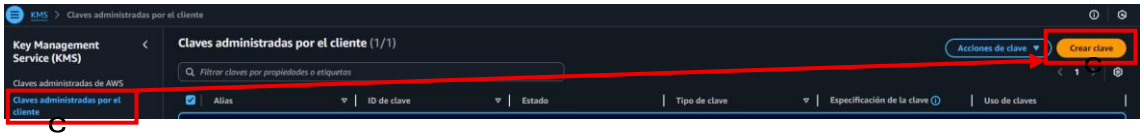


Darle un nombre a la instantánea

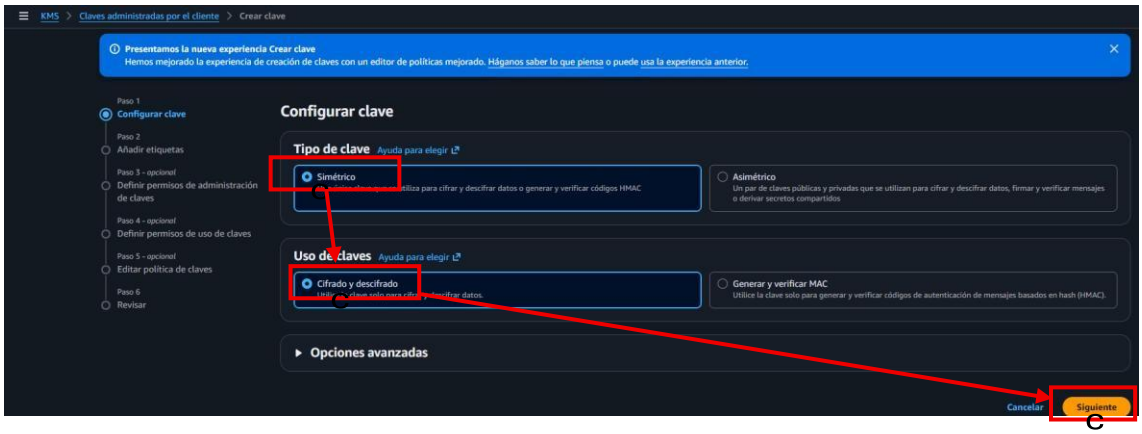
Cambiar KMS:

Debido a una política de AWS, como en un inicio utilizamos la clave de cifrado por defecto, podemos compartir la instantánea, pero en el nuevo laboratorio no podremos restaurar el RDS debido a que no permite descifrarlo con dicha clave por defecto, por lo que tenemos que crear una copia de la instantánea y cifrarla con una clave que crearemos a continuación:

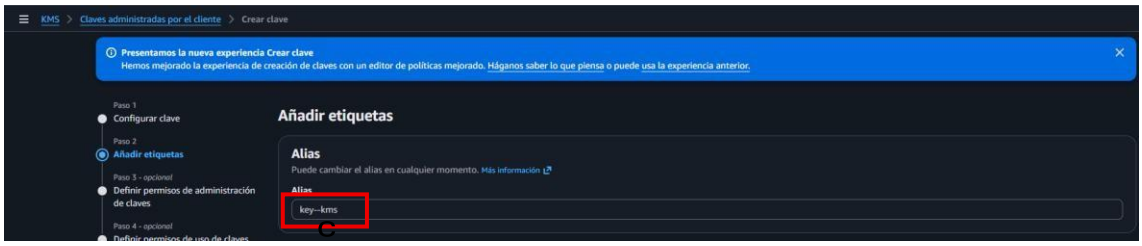
En AWS ir a KMS e ir al menú “Claves administradas por el cliente” y entrar en “Crear clave”



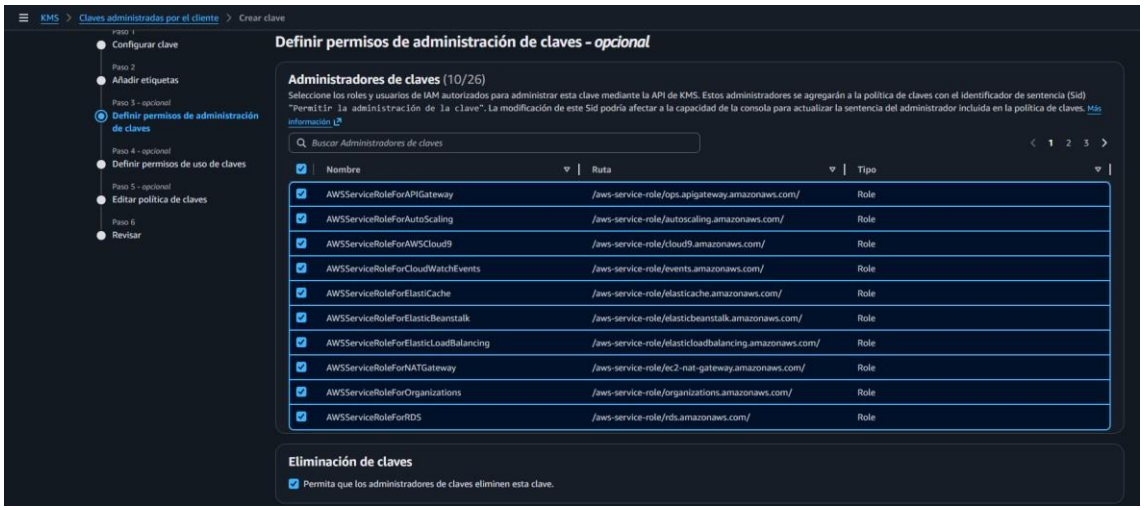
Elegir las opciones de “simétrico” y “cifrado y descifrado”



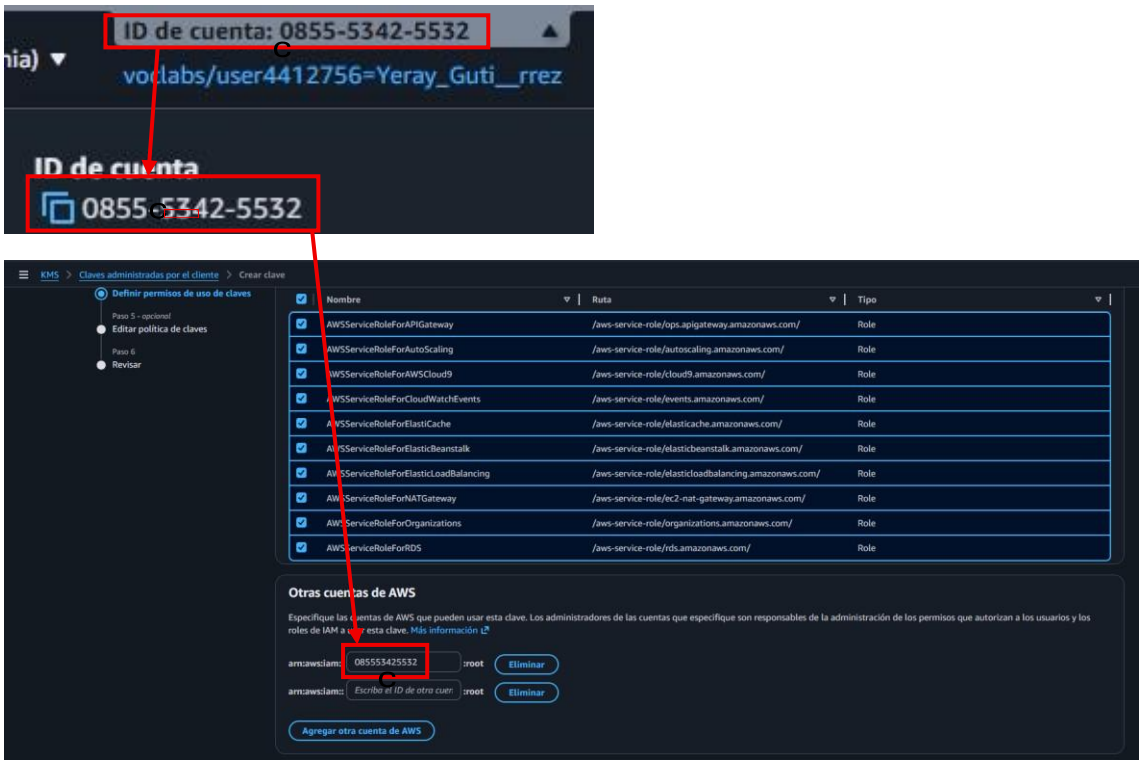
Darle un nombre a la clave:



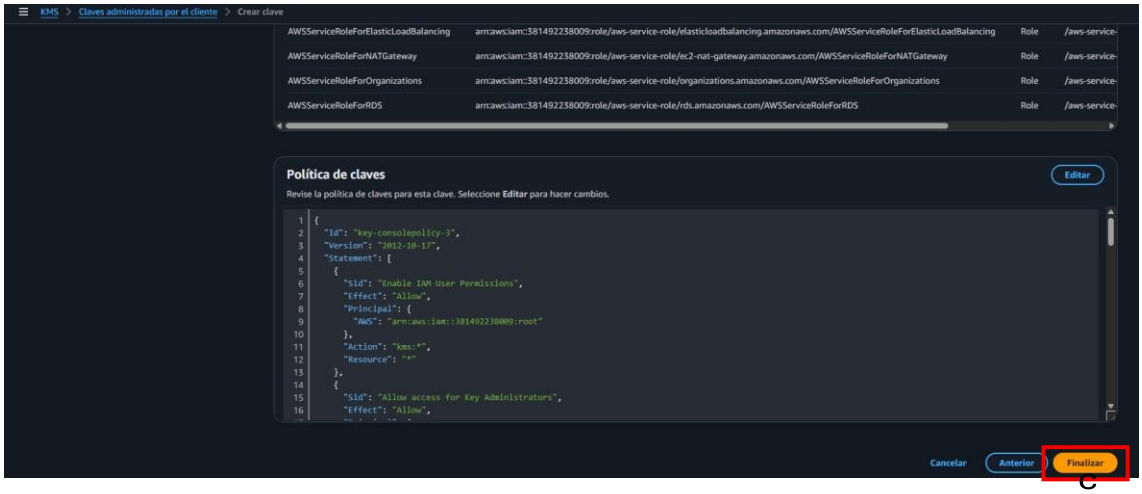
Darle permisos a los roles y usuarios para poder administrar esta clave:



Ahora, deberemos agregar el ID de la cuenta del nuevo laboratorio para permitirle usar la clave:



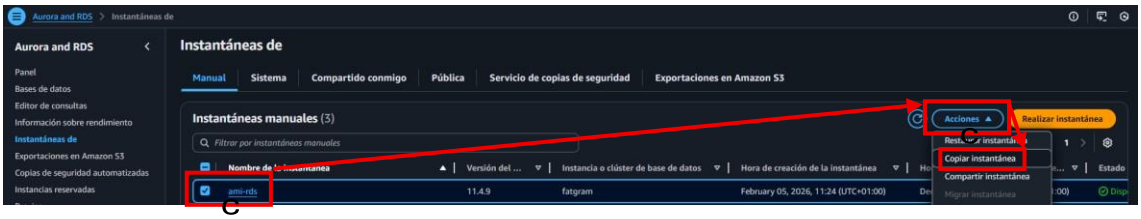
Ya podemos finalizar la creación de la clave:



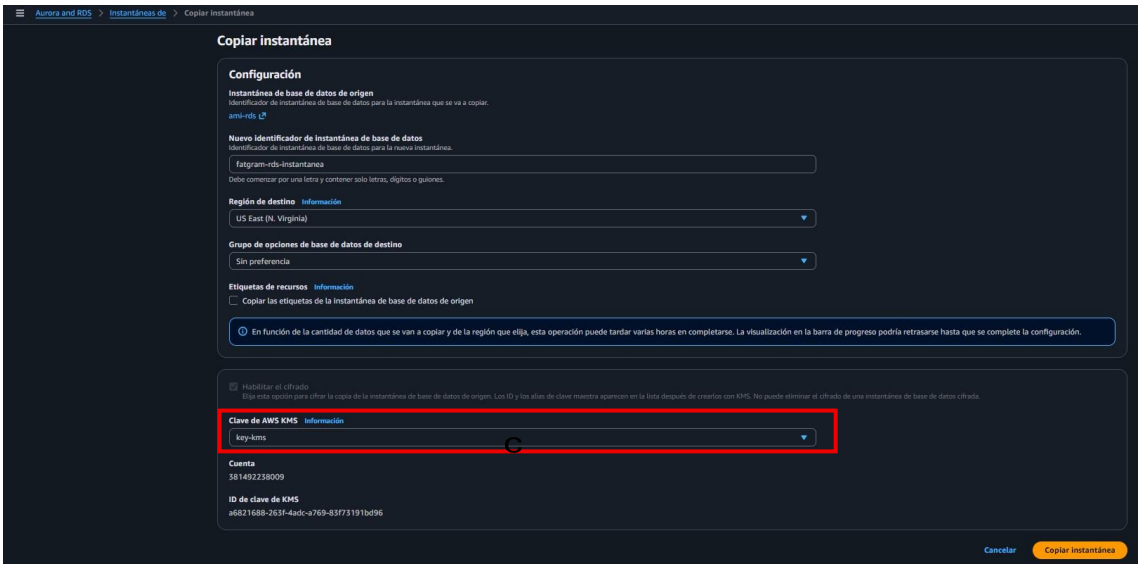
Una vez creada la clave, lo único que nos hace falta es el ARN de la clave:

Realizar copia de la instantánea:

Ya podemos realizar la copia de la instantánea del RDS para cifrarla con la nueva clave:

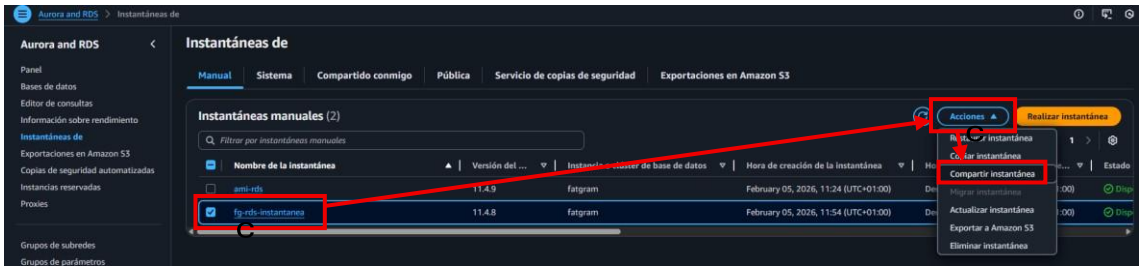


Debemos copiar la instantánea y en la opción de KMS, elegir la opción “usar un ARN clave” y elegir la clave creada:



Compartir la nueva instantánea:

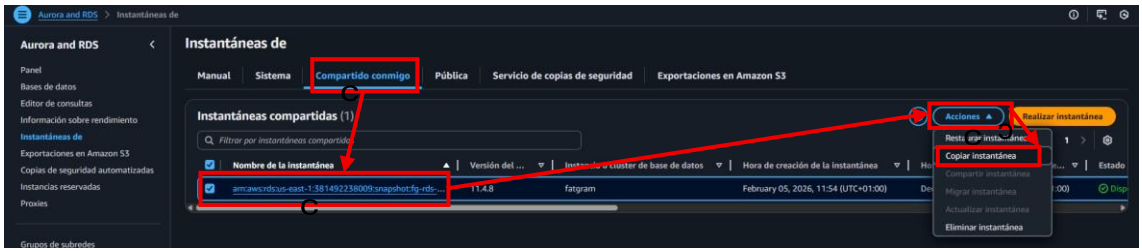
Una vez creada la copia, ir a Acciones>Compartir instantanea



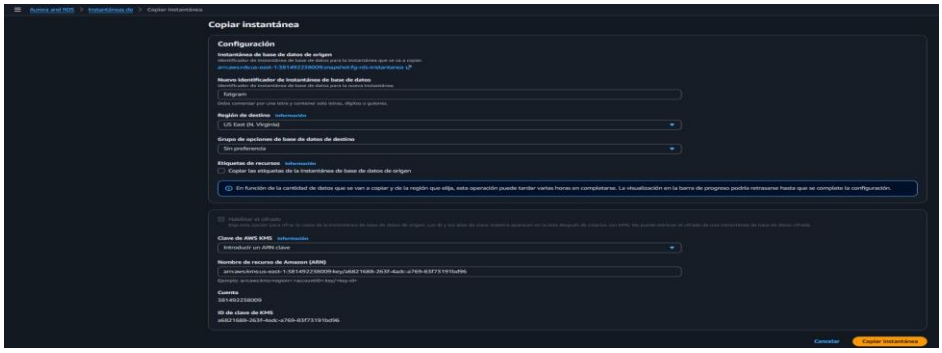
Aquí, añadir el ID de la cuenta del nuevo laboratorio y guardar

Copiar la instantánea en el nuevo servidor:

Ahora debemos ir al laboratorio nuevo y una vez más copiar la instantánea desde el apartado “Compartido conmigo”, esta vez para tenerla en local y poder descifrarla con la clave:

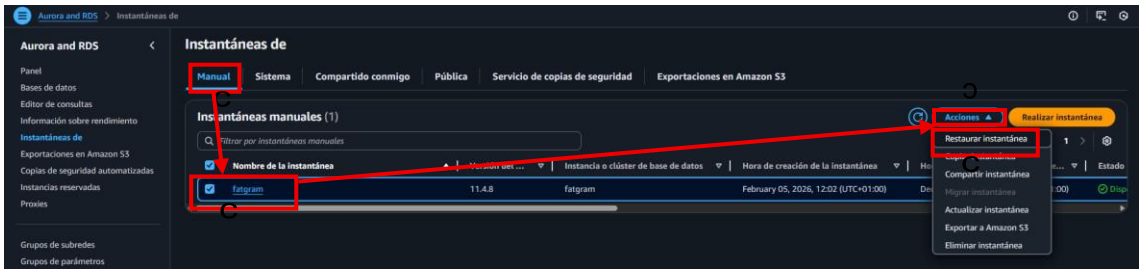


Copiar la instancia utilizando el ARN de la clave que hemos creado antes:

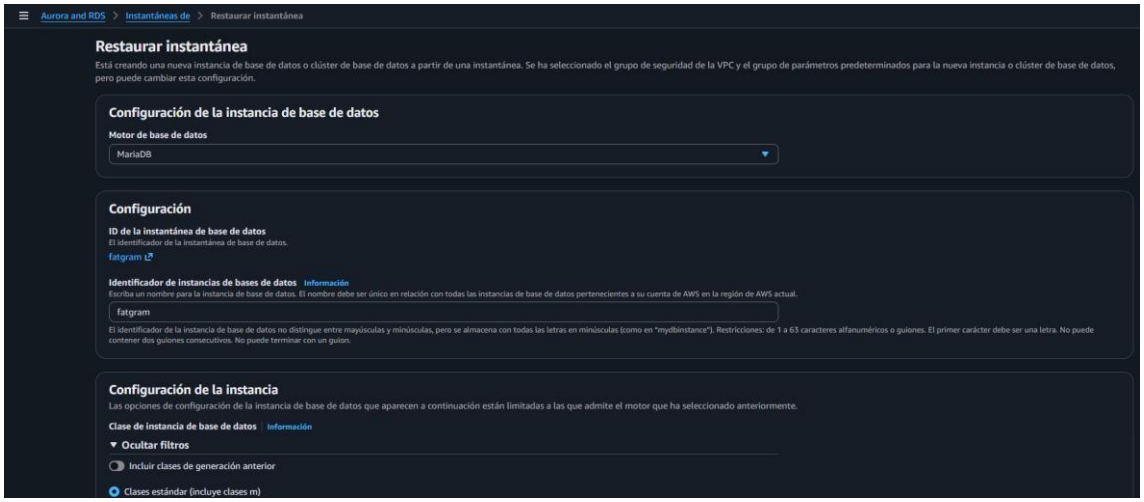


Restaurar el RDS:

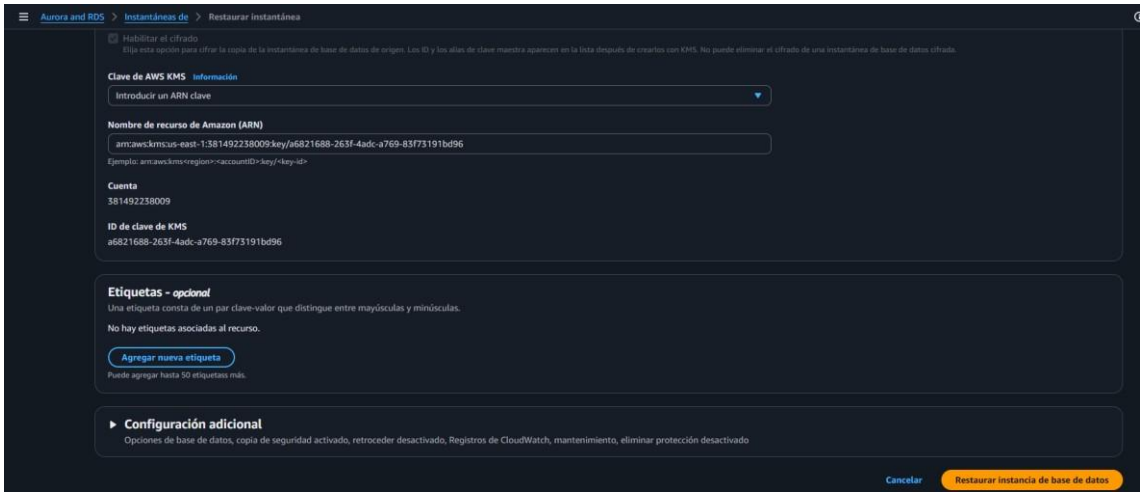
Ahora, ir a Acciones>Restaurar instantánea:



Debemosdarle un nombre al RDS



Y más abajo aparecerá la opción Clave de AWS KMS:



RDS migrado:

Y ya tendremos el RDS completamente funcional y con toda la base de datos

Monitorización

Creación de Usuario Remoto

```
monitor@ip-10-0-128-56:~$ whoami
monitor
monitor@ip-10-0-128-56:~$ cd .ssh/
monitor@ip-10-0-128-56:~/.ssh$ cat authorized_keys
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAABAQDvTVFLQDVU2HFSKYH131cVqZarlEeHcu0vht7I2bBLDVz66KlU9B6aPLGwcHicXGSuyVa+Ovb/Q06SG8hw0uuUvEN23diGV7XxP2l/5GVzI5WH5pBYZyq/Zh6
gNKv0ZGqBPF9333C9Z+Y3o9Cr0Fect05mQLeB33cq06xq2uc9fX0n73W8L9xiPGdTAT+2A0Vyr1LPmeMu1DAjF04yxkBV/BiiyBRJWC9eQBEcgYp5kK8Z6jtbQbx+AXFQvMVXjxvPU878xKTUeZ9FA2hLv7hV9R
rlllmpLQz9TbftKKmnIvnxCv8BM4IPvG0s827jUJz2igF/55DEnm/pVkvsYj2B
monitor@ip-10-0-128-56:~/.ssh$
```

Para empezar tenemos que crear un usuario en ambos servidores que servirán como usuario para realizar las conexiones SSH y poder monitorizar los servidores web.

Crearemos la carpeta .SSH y copiaremos la clave publica a el archivo de authorized_keys

```
monitor@ip-10-0-128-56:~/.ssh$ ls -al
total 12
drwx------ 2 monitor monitor 4096 Feb 11 08:03 .
drwxr-x--- 4 monitor monitor 4096 Feb 23 10:11 ..
-rw------- 1 monitor monitor  381 Feb 11 08:03 authorized_keys
monitor@ip-10-0-128-56:~/.ssh$
```

También nos aseguraremos de que el archivo tenga los permisos correctos

```
GNU nano 7.2                                    /etc/sudoers.tmp

# This fixes CVE-2005-4890 and possibly breaks some versions of kdesu
# (#1011624, https://bugs.kde.org/show_bug.cgi?id=452532)
Defaults        use_pty

# This preserves proxy settings from user environments of root
# equivalent users (group sudo)
#Defaults:%sudo env_keep += "http_proxy https_proxy ftp_proxy all_proxy no_proxy"

# This allows running arbitrary commands, but so does ALL, and it means
# different sudoers have their choice of editor respected.
#Defaults:%sudo env_keep += "EDITOR"

# Completely harmless preservation of a user preference.
#Defaults:%sudo env_keep += "GREP_COLOR"

# While you shouldn't normally run git as root, you need to with etckeeper
#Defaults:%sudo env_keep += "GIT_AUTHOR_* GIT_COMMITTER_*"

# Per-user preferences; root won't have sensible values for them.
#Defaults:%sudo env_keep += "EMAIL DEBEMAIL DEBFULLNAME"

# "sudo scp" or "sudo rsync" should be able to use your SSH agent.
#Defaults:%sudo env_keep += "SSH_AGENT_PID SSH_AUTH_SOCK"

# Ditto for GPG agent
#Defaults:%sudo env_keep += "GPG_AGENT_INFO"

# Host alias specification

# User alias specification

# Cmnd alias specification

# User privilege specification
root    ALL=(ALL:ALL) ALL

# Members of the admin group may gain root privileges
%admin ALL=(ALL) ALL

# Allow members of group sudo to execute any command
%sudo   ALL=(ALL:ALL) ALL

# See sudoers(5) for more information on "@include" directives:

@includedir /etc/sudoers.d

ubuntu ALL=(ALL) NOPASSWD: /bin/systemctl, /usr/bin/ss, /usr/bin/uptime, /usr/bin/free, /bin/df

monitor ALL=(ALL) NOPASSWD: ALL
```

Ahora dentro del archivo de Visudo, donde se definen los sudo añadiremos al código la línea donde Monitor, aquí definimos que Monitor pueda ejecutar cualquier comando como sudo y que no nos pidan contraseña de sudo

Fichero de Configuración

```
config
.ssh > config
Host proxy
  HostName 23.21.75.85
  User ubuntu
  Port 22
  IdentityFile "/home/ubuntu/.ssh/labsuser.pem"

Host monitorizacion
  HostName 52.54.178.148
  User ubuntu
  Port 22
  IdentityFile "/home/ubuntu/.ssh/labsuser.pem"

Host web
  HostName 10.0.128.56
  User monitor
  Port 22
  IdentityFile "/home/ubuntu/.ssh/labsuser.pem"
  ProxyJump proxy

Host web2
  HostName 10.0.128.32
  User monitor
  Port 22
  IdentityFile "/home/ubuntu/.ssh/labsuser.pem"
  ProxyJump proxy
```

Según la Configuración de nuestro script es muy importante el fichero Config de SSH, en la máquina de monitorización. Aquí establecemos mediante alias las conexiones SSH a los servidores web usando el usuario Monitor, ponemos la ruta donde se encuentra la clave privada y IP de cada máquina. Además, de la línea donde se define el proxy jump para que al hacer el SSH se pase por el Proxy para poder acceder a los servidores, ya que se encuentran en una subred privada.

Mas tarde explicaremos la importancia del fichero en el Script.

Pruebas de Conexiones SSH

Antes de cualquier prueba o ejecución no aseguramos que desde la instancia de monitorización se pueda hacer conexión SSH a los servidores web y al proxy.

WEB 1

```
ubuntu@ip-10-0-0-199:~/Monitorizacion$ ssh web
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.17.0-1007-aws x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

 System information as of Wed Feb 25 10:05:12 UTC 2026

  System load:  0.0                Temperature:           -273.1 C
  Usage of /:   63.0% of 13.49GB   Processes:             119
  Memory usage: 23%                Users logged in:       0
  Swap usage:   0%                 IPv4 address for ens5: 10.0.128.56

 * Ubuntu Pro delivers the most comprehensive open source security and
   compliance features.

   https://ubuntu.com/aws/pro

Expanded Security Maintenance for Applications is not enabled.

33 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

1 additional security update can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm


Last login: Wed Feb 25 09:46:26 2026 from 10.0.0.32
monitor@ip-10-0-128-56:~$
```

WEB 2

```
ubuntu@ip-10-0-0-199:~/Monitorizacion$ ssh web2
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.17.0-1007-aws x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

 System information as of Wed Feb 25 10:05:31 UTC 2026

  System load:  0.08               Temperature:           -273.1 C
  Usage of /:   63.6% of 13.49GB   Processes:             118
  Memory usage: 23%                Users logged in:       0
  Swap usage:   0%                 IPv4 address for ens5: 10.0.128.20

 * Ubuntu Pro delivers the most comprehensive open source security and
   compliance features.

   https://ubuntu.com/aws/pro

Expanded Security Maintenance for Applications is not enabled.

22 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

1 additional security update can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm


Last login: Wed Feb 11 09:58:17 2026 from 10.0.0.32
monitor@ip-10-0-128-20:~$
```

PROXY

```
ubuntu@ip-10-0-0-199:~/Monitorizacion$ ssh proxy
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.14.0-1018-aws x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

 System information as of Wed Feb 25 10:05:11 UTC 2026

  System load:  0.0                Temperature:           -273.1 C
  Usage of /:   49.8% of 13.49GB   Processes:             117
  Memory usage: 11%                Users logged in:       0
  Swap usage:   0%                 IPv4 address for ens5: 10.0.0.32

 * Ubuntu Pro delivers the most comprehensive open source security and
   compliance features.

   https://ubuntu.com/aws/pro

Expanded Security Maintenance for Applications is not enabled.

18 updates can be applied immediately.
13 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status


Last login: Mon Feb 23 08:12:55 2026 from 213.96.236.200
ubuntu@ip-10-0-0-32:~$
```

Creación y configuración del Script

Lenguaje Python

Hemos elegido crear el Script a partir de Python, ya que tiene muchas ventajas respecto al bash por ejemplo, como la implementación de librerías u otras utilidades.

Librerias y Configuración

monitor_servidores.py

monitor_servidores.py > ejecutar_comando

```python
import subprocess
import datetime
import os
import sys
import re


# ================ CONFIGURACIÓN ================

PUERTO_WEB = 80

# Alias tal como están en ~/.ssh/config
# None = ejecución local (sin SSH)
HOSTS = {
    "local": None,
    "web":   "web",
    "web2":  "web2"
}

BASE_DIR = os.path.dirname(os.path.abspath(__file__))
LOG_DIR  = os.path.join(BASE_DIR, "logs")
os.makedirs(LOG_DIR, exist_ok=True)

# Generar nombre de log con formato legible: monitor_YYYY-MM-DD_HH-MM-SS.log
timestamp = datetime.datetime.now().strftime("%Y-%m-%d_%H-%M-%S")
LOG_FILE = os.path.join(LOG_DIR, f"monitor_{timestamp}.log")
```

Para empezar, utilizaremos 5 librerías simples, sin acudir a ninguna externa:

1. **subprocess**: Permite ejecutar comandos del sistema. Se utiliza para realizar comprobaciones en los servidores (SSH, uso de CPU, memoria, disco, etc.).
2. **datetime**: Permite trabajar con fechas y horas. Lo utilizamos para añadir marcas de tiempo a los logs.
3. **os**: Gestiona archivos y directorios. Se utiliza para crear y organizar los archivos de logs.
4. **sys**: Permite acceder a argumentos del sistema. Se utiliza para diferenciar si el script se ejecuta de forma manual o automática.
5. **re**: Permite procesar texto con expresiones regulares. Se utiliza para limpiar los logs eliminando caracteres innecesarios.

En la parte de Configuracion establecemos los hosts a monitorizar, la ruta donde se generarán los logs y la configuración del formato fecha y hora de los logs.

Hosts (servidores)

En esta parte se define la lista de servidores que el script va a monitorizar.

Aquí ponemos exactamente los alias de los servidores que queremos monitorizar, exactamente igual que en el fichero config de SSH.

Logs

Definimos el directorio donde se van a guardar los logs de la monitorización.

Si el log existe se generará uno nuevo cada vez que se ejecute el script, sin borrar el anterior.

Esto permite tener un historial de lo que ha ocurrido en cada comprobación.

Fecha y hora del log

El script genera el nombre del archivo de log utilizando la fecha y la hora actual.

De esta forma:

- Cada ejecución crea un archivo diferente
- Se puede saber cuándo se realizó cada comprobación
- Se evitan sobre escrituras de archivos

Funciones Utilizadas para los Logs

```
# ================= FUNCIONES DE LOG =================

def quitar_colores(texto):
    """Elimina códigos ANSI de color para que el log quede limpio"""
    ansi_escape = re.compile(r'\x1B(?:[@-Z\\-_]|\[[0-?]*[ -/]*[@-~])')
    return ansi_escape.sub('', texto)


def log(mensaje):
    fecha = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    mensaje_limpio = quitar_colores(mensaje)
    with open(LOG_FILE, "a", encoding="utf-8") as f:
        f.write(f"[{fecha}] {mensaje_limpio}\n")


def mostrar(mensaje):
    print(mensaje)          # Con colores en terminal
    log(mensaje)            # Sin colores en archivo
```

En esta parte definimos las tres funciones que utilizaremos para la creación y configuración de los logs.

Quitar_Colores: está pensada para quitar los colores ASCI del log generado para evitar que salga un código raro, ya que los logs no traducen los códigos ASCI.

Log: está hecha para que en cada línea del log se muestre la fecha y la hora de del momento de ejecución.

Mostrar: se llama a print y log, para escribir los mensajes en el log y mostrarlos.

Función Ejecutar Comandos

```python
# ================ EJECUCIÓN DE COMANDOS ================

def ejecutar_comando(comando, host_alias, mostrar_error=True):
    try:
        if host_alias:
            cmd_final = ["ssh", "-o", "BatchMode=yes", "-o", "ConnectTimeout=10", host_alias, comando]
            shell = False
        else:
            cmd_final = comando
            shell = True

        resultado = subprocess.run(
            cmd_final,
            shell=shell,
            stdout=subprocess.PIPE,
            stderr=subprocess.PIPE,
            text=True,
            timeout=15,
            encoding="utf-8"
        )

        if resultado.returncode != 0:
            error_msg = resultado.stderr.strip() or "Error desconocido"
            if "Could not resolve hostname" in error_msg:
                error_msg = "No se pudo resolver el hostname (revisa ~/.ssh/config)"
            elif "Connection timed out" in error_msg or "No route to host" in error_msg:
                error_msg = "No se pudo conectar al servidor (timeout o no accesible)"
            elif mostrar_error:
                log(f"ERROR ejecutando '{comando}' en {host_alias or 'local'}: {error_msg}")

            return "", resultado.returncode, False, error_msg

        return resultado.stdout.strip(), resultado.returncode, True, ""

    except subprocess.TimeoutExpired:
        return "", 1, False, "Tiempo de espera agotado"
    except Exception as e:
        return "", 1, False, str(e)
```

Esta es una función donde se utilizan la librería del subprocess, esta es una función hecha para ejecutar comandos del sistema, en concreto el SSH a los servidores.

En la primera parte del código se realizará la conexión SSH a los servidores, definimos un tiempo de conexión de 10 segundos, se usará la librería subprocess.

En caso de no establecer conexión, se define una variable llamada resultado.returncode, que nos devuelve un 0 si es con éxito y si es distinto de cero devuelve distintos mensajes a modo de error, como no se pudo resolver hostname, connection time out, etc.

También se define un tiempo de espera de ejecución del comando, donde si se excede devolverá “tiempo de espera agotado”.

Monitoreo de Servidores

```python
# ================ COMPROBACIONES ================

def esta_puerto_abierto(host_alias, nombre):
    _, _, ok, error = ejecutar_comando(f"nc -z 127.0.0.1 {PUERTO_WEB}", host_alias, mostrar_error=False)
    if not ok and error:
        mostrar(f"[{nombre}] No se pudo comprobar puerto → {error}")
        return False
    estado = "sí" if ok else "no"
    mostrar(f"[{nombre}] Puerto {PUERTO_WEB} escuchando: {estado}")
    return ok


def obtener_respuesta_http(host_alias, nombre):
    cmd = f"curl -s -o /dev/null -w '%{{http_code}}' --max-time 8 http://127.0.0.1/"
    salida, _, ok, error = ejecutar_comando(cmd, host_alias, mostrar_error=False)

    if not ok:
        if error:
            mostrar(f"[{nombre}] No se pudo obtener respuesta HTTP → {error}")
        return "ERROR"

    if not salida.isdigit():
        return "ERROR"

    codigo = salida
    mostrar(f"[{nombre}] Respuesta HTTP: {codigo}")
    return codigo


def estado_servicio_web(host_alias, nombre):
    puerto_ok = esta_puerto_abierto(host_alias, nombre)
    http_code = obtener_respuesta_http(host_alias, nombre)

    if not puerto_ok:
        estado = "inactive (puerto cerrado)"
    elif http_code == "200":
        estado = "active"
    else:
        estado = f"inactive (HTTP {http_code})"

    mostrar(f"[{nombre}] Estado servicio web: {estado}")
    return estado == "active"
```

En esta parte del código se utilizará en todo el momento la función ejecutar comando, ya que son todos comandos del sistema. Se define una función por cada aspecto del sistema que queremos monitorear, como estado del servicio, puerto, CPU, RAM, etc.

Esta_puerto_abierto: en esta función se comprobará el estado del puerto, es decir se llama a la función ejecutar comando, para ejecutar el comando que consulta el estado del puerto del servidor web. Devolviendo error o en si el estado de este.

Obtener_respuesta_http: aquí se realiza la ejecución del curl para comprobar la respuesta http de los servidores web. Devuelve la respuesta http correcta o lanza un mensaje de error "no se ha obtenido respuesta http".

Estado_servicio_web: aquí se comprueba el estado del servicio instalado en nuestro caso es NGINX. Para comprobar el estado se comprueba el puerto y la respuesta HTTP, si estas fallan el servicio fallará y devolverá error Critico.

```
def obtener_rendimiento(host_alias, nombre):
    mostrar(f"\n[{nombre}] --- Rendimiento del equipo ---")

    # Carga CPU
    salida, _, _, error = ejecutar_comando("uptime", host_alias)
    load = "N/A"
    if salida and "load average:" in salida:
        try:
            load = salida.split("load average:")[1].split(",")[0].strip()
        except:
            pass
    mostrar(f"  Carga CPU (1 min): {load}")

    # RAM
    salida, _, _, _ = ejecutar_comando("free -m", host_alias)
    uso_ram = "N/A"
    for linea in salida.splitlines():
        if linea.startswith("Mem:"):
            campos = linea.split()
            if len(campos) >= 7:
                try:
                    total = int(campos[1])
                    usada = int(campos[2]) + int(campos[3])  # used + buffers/cache approx
                    if total > 0:
                        uso_ram = round((usada / total) * 100, 1)
                except:
                    pass
    mostrar(f"  Uso RAM: {uso_ram}%")

    # Disco /
    salida, _, _, _ = ejecutar_comando("df -h / | tail -1 | awk '{print $5}'", host_alias)
    disco = salida.rstrip('%') if salida else "N/A"
    mostrar(f"  Uso disco /: {disco}%")
```

Obtener_Rendimiento: en esta función se comprueba el rendimiento del servidor, comprobamos CPU, RAM y capacidad de Disco. Mostrando porcentaje de la CPU en uso, RAM, y la capacidad del Disco, si algunos de estos están al máximo lanzara mensaje de error Crítico.

Monitorización Completa

```python
# ================ MONITORIZACIÓN COMPLETA ================

def monitor_completo(nombre, host_alias):
    print("\n" + "=" * 70)
    mostrar(f" MONITOR COMPLETO → {nombre.upper()} ({host_alias or 'LOCAL'})")
    print("-" * 70)

    activo = estado_servicio_web(host_alias, nombre)
    obtener_rendimiento(host_alias, nombre)

    resultado = "OK" if activo else "CRÍTICO"
    color = "\033[92m" if activo else "\033[91m"
    reset = "\033[0m"

    # En pantalla con color
    print()
    print(f"RESULTADO FINAL: {color}{resultado}{reset}")

    # En log sin color + formato más claro
    log(f"RESULTADO FINAL: {resultado}")

    print("=" * 70 + "\n")
```

En esta parte configuraremos la ejecución de la monitorización completa, este será una opción localizada en el menú, donde se realizará la monitorización completa del servidor, sin tener que elegir opciones dentro del monitoreo.

Aquí se llamará a todas las funciones definidas en la parte de comprobación del sistema y se ejecutaran todas. Imprimiendo el resultado en el log.

Y devolviendo el resultado final ya sea OK o CRÍTICO, en color verde o rojo.

Modo Automático (CRON)

```python
# ================ MODO AUTOMÁTICO (CRON) ================

def modo_automatico():
    """Ejecuta monitorización de todos los hosts sin interacción"""
    print("\n=== MODO AUTOMÁTICO (CRON) ===\n")
    log("Modo automático iniciado")

    for nombre, host in HOSTS.items():
        try:
            monitor_comp
        except Exception
            error = f"[{nombre}] ERROR en monitorización: {e}"
            print(error)
            log(error)

    print("\n=== FIN MODO AUTOMÁTICO ===\n")
    log("Modo automático finalizado")
```

(variable) nombre: str

Loading...

Aquí configuramos la función para poder ejecutar el cron de manera automática, esta función define la ejecución del script de manera automática según la configuración del cron, esta monitorización será de todas las maquinas.

Menús

```python
# ================ MENÚS ================

def menu_principal():
    print("""
=== MONITOR CENTRAL ===
1. Seleccionar máquina
2. Monitorizar TODAS
3. Salir
""")


def menu_maquina(nombre):
    print(f"""
--- {nombre.upper()} ---
1. Rendimiento del equipo
2. Estado Nginx (puerto + HTTP)
3. Comprobar puerto {PUERTO_WEB}
4. Comprobar respuesta HTTP
5. Monitorización completa
6. Volver
""")


def seleccionar_maquina():
    nombres = list(HOSTS.keys())
    for i, n in enumerate(nombres, 1):
        print(f"{i}. {n}")

    while True:
        try:
            sel = int(input("\nSelecciona número: "))
            if 1 <= sel <= len(nombres):
                return nombres[sel-1]
            print(f"Elige entre 1 y {len(nombres)}")
        except ValueError:
            print("Introduce un número válido")
```

Ahora configuramos los dos menús el de inicio donde elegimos la maquina a monitorizar o todas las maquinas, en el otro menú estará compuesto por toda la monitorización del servidor, puerto, respuesta http, rendimiento, etc.

Y la última función está pensada para que el script interprete las opciones elegidas al meter números, por ejemplo, al inicio, hay 3 opciones: al meter un 1 elegimos la primera opción.

Modo Interactivo

```python
# ================= MODO INTERACTIVO =================

def modo_manual():
    while True:
        menu_principal()
        opcion = input("Opción: ").strip()

        if opcion == "1":
            nombre = seleccionar_maquina()
            host = HOSTS[nombre]

            while True:
                menu_maquina(nombre)
                sub_op = input("Opción: ").strip()

                if sub_op == "1":
                    obtener_rendimiento(host, nombre)
                elif sub_op == "2":
                    estado_servicio_web(host, nombre)
                elif sub_op == "3":
                    esta_puerto_abierto(host, nombre)
                elif sub_op == "4":
                    obtener_respuesta_http(host, nombre)
                elif sub_op == "5":
                    monitor_completo(nombre, host)
                elif sub_op == "6":
                    break
                else:
                    print("Opción no válida")

                print()

        elif opcion == "2":
            print("\nMonitorizando todas las máquinas...\n")
            for nom, h in HOSTS.items():
                monitor_completo(nom, h)
            input("\nPulse Enter para continuar...")

        elif opcion == "3":
            print("Saliendo...")
            break

        else:
            print("Opción no válida")
```

En esta parte configuramos, el modo manual, donde hacemos que se ejecuten todas las funciones anteriores para monitorear todos los aspectos, según el servidor elegido. 6 opciones en el menú de monitoreo y 3 opciones en el menú inicial.

Inicio de Script

```python
# ================= INICIO =================

if __name__ == "__main__":
    print("Iniciando monitor de servidores...")
    log("Script iniciado")

    try:
        # 👉 modo cron
        if len(sys.argv) > 1 and sys.argv[1] == "auto":
            modo_automatico()
        else:
            modo_manual()

    except KeyboardInterrupt:
        print("\n\nInterrumpido por el usuario.")
    except Exception as e:
        print(f"\nError inesperado: {e}")
        log(f"ERROR CRÍTICO: {str(e)}")
    finally:
        log("Script finalizado")
```

Para terminar, establecemos el inicio de la ejecución del script, es decir, los mensajes que mostrara. También establecemos el parámetro con el que ejecutaremos el cron de manera automática, en nuestro caso auto. También lanzamos mensajes al interrumpir el script, errores, script finalizado, etc.

Configuración de CRON

```
  GNU nano 7.2                                        /tmp/crontab.5Sx5UH/crontab
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h  dom mon dow   command

0 */1 * * * /usr/bin/python3 /home/ubuntu/Monitorizacion/monitor_servidores.py auto >> /home/ubuntu/Monitorizacion/logs/cron.log 2>&1
```

Para configurar el CRON, tenemos que primero instalar el paquete del CRON.

Después con crontab -e abriremos la tabla del cron donde tenemos que añadir una línea del estilo de la imagen. En nuestro caso los primeros caracteres, son el tiempo de la configuración del cron, en nuestro caso lo tenemos cada hora, pero en la foto podemos ver que este cada minuto, para comprobar errores.

Seguido definimos la ruta donde esta almacenado el script, seguido del parámetro definido en el script, auto.

```
ubuntu@ip-10-0-0-199:~/Monitorizacion$ crontab -l
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h  dom mon dow   command

0 */1 * * * /usr/bin/python3 /home/ubuntu/Monitorizacion/monitor_servidores.py auto >> /home/ubuntu/Monitorizacion/logs/cron.log 2>&1
ubuntu@ip-10-0-0-199:~/Monitorizacion$
```

Después con crontab -l comprobamos si hay alguna tabla instalada, veremos que esta la que acabamos de definir.

8. Seguimiento

En nuestro proyecto, el seguimiento del sistema es una parte fundamental para asegurar que todos los servicios funcionan correctamente y se mantienen disponibles en todo momento. Dado que la aplicación debe estar operativa 24/7, hemos definido un conjunto de tareas de control y revisión que nos permiten detectar posibles fallos, aplicar soluciones y mejorar el rendimiento de la infraestructura.

Para organizar este seguimiento, hemos establecido diferentes niveles de control en función del tiempo. De esta forma, podemos revisar el sistema de manera constante sin sobrecargar el trabajo diario.

Por un lado, realizamos un **seguimiento diario**, centrado en comprobar que todo funciona correctamente a nivel general. En este nivel revisamos principalmente:

- Los logs generados automáticamente por la monitorización
- El estado de los servicios web
- El acceso a la aplicación desde el punto de vista del usuario

Además, de forma **semanal**, llevamos a cabo una revisión más completa del sistema, donde analizamos aspectos más técnicos y de rendimiento. En estas revisiones comprobamos:

- El uso de recursos (CPU, memoria y almacenamiento)
- El estado general de los servidores
- La correcta realización de las copias de seguridad
- La existencia de errores repetitivos en el sistema

Por último, realizamos un **seguimiento mensual**, en el que evaluamos el funcionamiento global del proyecto. En este punto analizamos el rendimiento general, el consumo de recursos en la nube y posibles mejoras que se puedan aplicar para optimizar el sistema.

Para medir la calidad del servicio, nos apoyamos en varios indicadores que nos permiten tener una visión clara del estado del sistema. Entre los más importantes se encuentran la disponibilidad del servicio, el tiempo de respuesta de la aplicación y el número de incidencias detectadas. Estos indicadores nos ayudan a identificar problemas antes de que afecten a los usuarios.

En cuanto a la gestión de incidencias, seguimos un proceso organizado. Cuando detectamos un problema, ya sea mediante la monitorización o de forma manual, seguimos los siguientes pasos:

- Identificamos y registramos la incidencia
- Analizamos la causa del problema
- Aplicamos una solución
- Verificamos que el sistema vuelve a funcionar correctamente

De esta forma, mantenemos un control claro de los fallos y evitamos que se repitan en el futuro. Además, cuando realizamos cambios en la infraestructura o en la configuración, intentamos hacerlo en momentos de menor uso del sistema para evitar afectar a los usuarios.

También consideramos importante la participación de los usuarios, ya que nos pueden aportar información valiosa sobre el funcionamiento de la aplicación. Por ello, recogemos posibles errores, sugerencias o mejoras a través de los medios disponibles y las analizamos para seguir mejorando el sistema.

Durante el seguimiento pueden aparecer desviaciones, como caídas del servicio o problemas de rendimiento. En estos casos, actuamos aplicando medidas correctivas, como ajustes en la configuración, optimización de recursos o revisión de los servicios afectados. Todas estas acciones quedan registradas para poder consultarlas en el futuro.

Para mantener un control completo del sistema, documentamos toda la actividad relacionada con el seguimiento. Esto incluye:

- Registros de logs
- Incidencias detectadas
- Cambios realizados en la infraestructura
- Informes periódicos del estado del sistema

Esta documentación nos permite tener un historial del proyecto y facilita tanto el mantenimiento como futuras mejoras.

Finalmente, el mantenimiento del sistema lo dividimos en dos tipos: **mantenimiento preventivo y mantenimiento correctivo**, con el objetivo de garantizar la estabilidad y continuidad del servicio.

Por un lado, el **mantenimiento preventivo** consiste en realizar acciones periódicas para evitar fallos antes de que ocurran. En nuestro caso, llevamos a cabo tareas como:

- Actualización de sistemas, servicios y dependencias
- Revisión de la seguridad y de los accesos
- Verificación del correcto funcionamiento de las copias de seguridad
- Optimización del rendimiento de los servidores

Por otro lado, el **mantenimiento correctivo** se aplica cuando ya ha ocurrido una incidencia o fallo en el sistema. En estos casos, nos centramos en restaurar el servicio lo antes posible, realizando acciones como:

- Reinicio de servicios o servidores
- Corrección de errores en la configuración
- Recuperación de datos a partir de copias de seguridad
- Ajustes de recursos en la infraestructura

Gracias a este conjunto de procedimientos de seguimiento y mantenimiento, conseguimos asegurar que el sistema se mantenga estable, seguro y disponible, adaptándose a las necesidades del proyecto y de los usuarios.

9. Conclusiones y Valoración Personal

Valoración del trabajo en grupo

En general, pensamos que el trabajo en grupo ha sido bueno. Hemos sabido organizarnos y repartir las tareas entre los tres para poder avanzar en el proyecto. Cada uno ha trabajado en distintas partes y nos hemos ido ayudando cuando alguno tenía dudas.

También hemos tenido algunos problemas, sobre todo a la hora de organizarnos o coincidir para trabajar todos a la vez. A veces nos ha costado comunicarnos o llevar el mismo ritmo, pero al final hemos conseguido adaptarnos y sacar el trabajo adelante.

Creemos que trabajar en grupo nos ha servido para aprender a colaborar, tener paciencia y entender que no todos trabajamos igual, algo que seguramente nos servirá en el futuro.

Reflexión final del proyecto

Este proyecto nos ha servido para aplicar muchas cosas que hemos aprendido durante el curso, pero de una forma más práctica. Hemos podido ver cómo se monta un proyecto desde cero y todo lo que hay detrás, no solo hacerlo funcionar, sino también organizarlo y documentarlo.

Aunque al principio parecía complicado, poco a poco hemos ido entendiendo mejor cómo funciona todo y eso nos ha dado más confianza. Nos ha ayudado a aprender por nuestra cuenta, buscar soluciones y no rendirnos cuando algo no salía.

Dificultades encontradas

Durante el proyecto hemos tenido bastantes dificultades. En muchos momentos las cosas no funcionaban como esperábamos y no sabíamos muy bien por qué. Esto nos ha hecho perder bastante tiempo intentando encontrar errores.

También hemos tenido problemas con la organización del tiempo y con algunos cambios que hemos tenido que hacer durante el proyecto, como mover cosas de un sitio a otro o repetir configuraciones.

Aun así, creemos que estas dificultades nos han ayudado a aprender más, porque hemos tenido que investigar, probar y equivocarnos hasta encontrar la solución.

Calidad del trabajo realizado y aprendizajes

Creemos que el resultado final es bueno, ya que hemos conseguido que el proyecto funcione y cumpla con lo que se pedía. Aunque hay cosas que se podrían mejorar, estamos contentos con el trabajo realizado.

Sobre todo, hemos aprendido a trabajar de forma más autónoma, a organizarnos mejor y a resolver problemas. También hemos visto cómo se hace un proyecto más completo, no solo una práctica puntual.

Qué mejoraríamos en futuras ocasiones

Si tuviéramos que repetir el proyecto, intentaríamos organizarnos mejor desde el principio y repartir las tareas de forma más clara. También intentaríamos ir haciendo las cosas poco a poco y no dejar tanto trabajo para el final.

Además, creemos que sería importante probar más lo que hacemos para detectar errores antes y no tener problemas al final.

10. Bibliografía y Anexos

Soporte NGINX: https://nginx.org/

Soporte AWS: https://docs.aws.amazon.com/

Gemini: https://gemini.google.com/?hl=es-ES

Soporte WordPress: https://wordpress.com/es/support/

Documentación de Python: https://docs.python.org/3/